

CSS syntax & selectors

CSS syntax & selectors

بناء كود CSS + إزاي نختار العناصر

يعني إيه CSS؟

CSS = **Cascading Style Sheets**

هي اللغة المسؤولة عن **شكل الصفحة**:

- ألوان
- خطوط
- مسافات
- Layout
- Animation

HTML = structure

CSS = appearance

JS = behavior

CSS Syntax (شكل كود CSS)

الشكل العام لأي كود CSS:

```
selector {  
  property: value;  
}
```

مثال:

```
p {  
  color: red;  
  font-size: 18px;  
}
```

- **Selector** → *p* (العنصر الذي عايزين نغير شكله)
- *color* → Property
- *red* → Value
- *;* → نهاية السطر
- *{ }* → حدود القواعد الخاصة بالعنصر

كل Rule مستقلة عن الثانية

طرق كتابة CSS

1 Inline CSS ❌ (غير مفضل)

```
<p style="color: red;">Hello</p>
```

- صعب الصيانة
- يفسر فكرة التنظيم

2 Internal CSS ⚠️ (للتجارب فقط)

```
<style>
p {
  color: blue;
}
</style>
```

3 External CSS ✅ (الصحيح)

```
<link rel="stylesheet" href="style.css">
```

```
p {
  color: green;
}
```

- قابل لإعادة الاستخدام
- منظم
- احترافي

CSS Selectors (إزاي نختار العناصر)

Element Selector¹

```
p {  
  color: red;  
}
```

- بيختار كل عناصر `<p>`

Class Selector² (الأكثر استخدامًا)

```
.card {  
  background-color: #f5f5f5;  
}
```

```
<div class="card"></div>
```

- يبدأ بنقطة .
- ينفع يتكرر على أكثر من عنصر

ID Selector³ (بحذر)

```
#main-title {  
  font-size: 32px;  
}
```

```
<h1 id="main-title"></h1>
```

- يبدأ بـ #
- عنصر واحد فقط
- مش مفضل في الـ styling الكثير

Grouping Selectors⁴

```
h1, h2, h3 {  
  font-family: Arial;  
}
```

Descendant Selector⁵

```
nav a {  
  color: white;  
}
```

Direct Child Selector⁶

```
ul > li {  
  list-style: none;  
}
```

Attribute Selector⁷

```
input[type="email"] {  
  border: 2px solid blue;  
}
```

Universal Selector

```
*{  
  margin: 0;  
  padding: 0;  
}
```

Selector Priority (مين يكسب؟)

من الأقوى للأضعف:

1. **important!**

2. Inline

3. ID

4. Class

5. Element

كل ما الـ selector كان أدق، قوته تزيد

Best Practices

استخدم Classes بدل IDs 

خلي أسماء الـ classes واضحة 

```
/* صح */  
.product-card
```

```
/* غلط */
```

```
.box1
```

تجنب **important!** ✓

خلي الكود readable ومنظم ✓

Common Mistakes

نسيان ; ✗

كتابة class من غير . ✗

استخدام ID لكل حاجة ✗

خط CSS مع HTML inline ✗

أسماء classes ملهاش معنى ✗

Task صغير

- اعمل ملف **style.css**

- غير:

- لون كل **<p>**

- ستايل class اسمها **box**.

- عنصر واحد بـ ID

● استخدم:

Element selector ○

Class selector ○

Descendant selector ○

What is CSS?

CSS = Cascading Style Sheets

It's the language responsible for the page's appearance:

Colors

Fonts

Spacing

Layout

Animation

HTML = structure

CSS = appearance

JS = behavior

CSS Syntax (CSS Code Structure)

The general structure of any CSS code:

```
selector {  
    property: value;  
}
```

Example:

```
p {  
  color: red;  
  font-size: 18px;  
}
```

p → Selector (the element we want to style)

color → Property

red → Value

; → end of the line

{ } → boundaries of the rules for the element

Each rule is independent from the others

Ways to Write CSS

❶ Inline CSS (Not Recommended)

```
<p style="color: red;">Hello</p>
```

Hard to maintain

Breaks organization

❏ Internal CSS (For Testing Only)

```
<style>
  p {
    color: blue;
  }
</style>
```

❏ External CSS (Correct Way)

```
<link rel="stylesheet" href="style.css">

p {
  color: green;
}
```

Reusable

Organized

Professional

CSS Selectors (How to Select Elements)

1 Element Selector

```
p {  
  color: red;  
}
```

Selects all `<p>` elements

2 Class Selector (Most Used)

```
.card {  
  background-color: #f5f5f5;  
}
```

```
<div class="card"></div>
```

Starts with a dot `.`

Can be applied to multiple elements

3 ID Selector (Use Carefully)

```
#main-title {  
    font-size: 32px;  
}
```

```
<h1 id="main-title"></h1>
```

Starts with #

Only one element

Not recommended for heavy styling

④ Grouping Selectors

```
h1, h2, h3 {  
    font-family: Arial;  
}
```

⑤ Descendant Selector

```
nav a {  
    color: white;  
}
```

6 Direct Child Selector

```
ul > li {  
    list-style: none;  
}
```

7 Attribute Selector

```
input[type="email"] {  
    border: 2px solid blue;  
}
```

8 Universal Selector

```
* {  
    margin: 0;  
    padding: 0;  
}
```

Selector Priority (Who Wins?)

From strongest to weakest:

!important

Inline

ID

Class

Element

The more specific the selector, the stronger it is

Best Practices

✓ Use Classes instead of IDs

✓ Make class names clear

/ Correct */*

.product-card

/ Wrong */*

.box1

- ✓ Avoid `!important`
- ✓ Keep code readable and organized

Common Mistakes

- ✗ Forgetting `;`
- ✗ Writing a class without `.`
- ✗ Using ID for everything
- ✗ Mixing CSS with inline HTML
- ✗ Meaningless class names

Small Task

Create a `style.css` file

Change:

- Color of all `<p>`
- Style a class named `.box`
- One element with an ID

Use:

- Element selector
- Class selector

- Descendant selector

Specificity & Cascade

Specificity & Cascade

خلينا نبدأ بسؤال مهم:

ليه الـ CSS اللي أنا كاتبه مش بيتطبق؟!

الإجابة غالبًا: Cascade أو Specificity

أولًا: يعني إيه Cascade؟

Cascade = ترتيب تنفيذ قواعد الـ CSS

المتصفح ييشوف كذا Rule، ويسأل:

أطبق أنهي واحد؟

بيمشي بالترتيب ده

1) Specificity

2) Order (اللي جاي بعد يكسب)

3) **important!** (آخر الحلول)

Specificity (قوة الاختيار)

كل Selector له قوة
الأقوى هو اللي بيتنفذ.

ترتيب القوة (من الأضعف للأقوى):

Element < Class < ID < Inline Style

جدول سريع

القوة	Example	Selector
ضعيفة	p	Element
متوسطة	text.	Class
قوية	title#	ID
قوية جدًا	" "=style	Inline
أقوى حاجة		!important

أمثلة عملية

مثال 1

```
p {  
  color: red;  
}
```

```
.text {  
  color: blue;  
}
```

```
<p class="text">Hello</p>
```

✓ اللون هيبقى أزرق

ليه؟

Class أقوى من Element

مثال 2

```
#main p {  
  color: green;  
}
```

```
p {  
  color: red;  
}
```

```
<div id="main">  
  <p>Hello</p>  
</div>
```

✓ اللون أخضر

ليه؟

ID + Element أقوى

مثال 3 (Order)

```
.text {  
  color: red;  
}
```

```
.text {  
  color: blue;  
}
```

✓ اللون أزرق

ليه؟

نفس القوة → اللي تحت يكسب

important! (بحذرا!)

```
p {  
  color: red !important;  
}
```

```
.text {  
  color: blue;  
}
```

```
<p class="text">Hello</p>
```

اللون أحمر ✓

استخدمه:

- في حالات نادرة
- مش كل دائم

قاعدة ذهبية

كل ما ال selector أبسط، الكود أنصف ✨

وحش: ✗

```
body div.main ul li a span.text {  
  color: red;
```



```
}
```

✓ أحسن:

```
.text {  
  color: red;  
}
```

Common Bugs

- ليه الـ CSS مش شغال؟
 - Selector أضعف
 - Rule اتكتب قبل Rule أقوى
 - Inline أو **important!** كاسره
 - CSS ملفه متحمل متأخر

نصائح Frontend حقيقية

- اعتمد على **Classes**
- قلّل استخدام IDs في الـ styling
- بلاش **important!** إلا مضطر

● افتح **Styles** → **DevTools** وشوف مين كسب وليه

اختر الإجابة الصحيحة (MCQ)

1 أي Selector أقوى؟

- A) `p`
- B) `.text`
- C) `#title`
- D) `div p`

2 لو Rule بنفس القوة، مين يكسب؟

- (A) الأول
- (B) الأخير
- (C) اللي فيه Class
- (D) اللي فيه Element

3!important يُستخدم:

(A) دائمًا

(B) لتحسين الأداء

(C) كحل أخير

(D) بـدل Classes

Let's start with an important question:

Why isn't the CSS I wrote being applied?!

The answer is usually: **Specificity** or **Cascade**

First: What is Cascade?

Cascade = the order of executing CSS rules

The browser sees multiple rules and asks:

Which one should I apply?

It follows this order:

- 1 Specificity
- 2 Order (the one that comes later wins)
- 3 Important (last resort)

Specificity (Selector Strength)

Each selector has a strength

The strongest one gets applied.

Strength order (from weakest to strongest):

Element < Class < ID < Inline Style

Quick Table:

Selector	Example	Strength
Element	p	Weak
Class	.text	Medium
ID	#title	Strong
Inline	style=""	Very Strong
!important	—	Strongest

Practical Examples

Example 1

```
p {
  color: red;
}
```

```
.text {  
  color: blue;  
}
```

```
<p class="text">Hello</p>
```

✓ The color will be **blue**

Why? Class is stronger than Element

Example 2

```
#main p {  
  color: green;  
}
```

```
p {  
  color: red;  
}
```

```
<div id="main">
  <p>Hello</p>
</div>
```

✓ The color will be **green**

Why? ID + Element is stronger

Example 3 (Order)

```
.text {
  color: red;
}
```

```
.text {
  color: blue;
}
```

✓ The color will be **blue**

Why? Same strength → the one below wins

!important (Use Carefully!)

```
p {  
  color: red !important;  
}
```

```
.text {  
  color: blue;  
}
```

```
<p class="text">Hello</p>
```

✓ The color will be **red**

Use it:

- In rare cases
- Not a permanent solution

Golden Rule

The simpler the selector, the cleaner the code

✗ Bad:

```
body div.main ul li a span.text {  
    color: red;  
}
```

✓ Better:

```
.text {  
    color: red;  
}
```

Common Bugs

Why isn't CSS working?

- Selector is weaker
- Rule is written before a stronger rule
- Inline or !important breaking it

- CSS file loaded late

Real Frontend Tips

- Rely on Classes
- Reduce using IDs in styling
- Avoid !important unless necessary
- Open DevTools → Styles and see who won and why

Choose the Correct Answer (MCQ)

1 Which selector is strongest?

- A) p
- B) .text
- C) #title
- D) div p

2 If rules have the same strength, who wins?

- A) The first
- B) The last
- C) The one with Class
- D) The one with Element

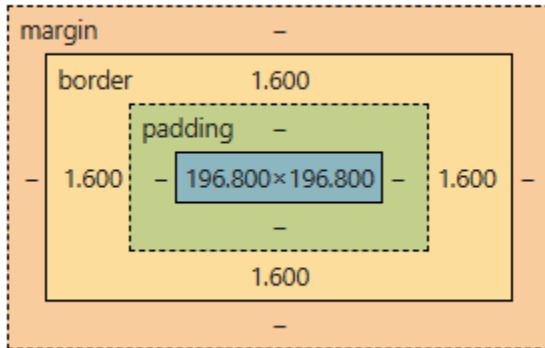
3) Important is used:

- A) Always
- B) For performance improvement
- C) As a last resort
- D) Instead of Classes

Box model

Box model

إزاي المتصفح يحسب حجم أي عنصر؟



يعني إيه Box Model؟

أي عنصر في الصفحة المتصفح

بیشوفه ک صندوق (Box)

الصندوق ده مكوّن من 4 طبقات:

أي مشكلة Layout بنسبة 70% سببها

فهم غلط لـ Box Model

Content 1

ده المحتوى الحقيقي:

- Text
- Image
- Video

```
.box {
```

```
width: 200px;
```

```
height: 100px;
```

```
}
```

العرض والطول يطبقوا على **المحتوى فقط** (افتراضيًا)

2 Padding (المسافة الداخلية)

المسافة بين المحتوى و border

```
.box {
```

```
padding: 20px;
```

```
}
```

أو بشكل منفصل:

```
padding: 10px 20px;
```

```
padding-top: 10px;
```

```
padding-right: 20px;
```

بيكر حجم العنصر

Border 3

الإطار اللي حوالين العنصر

```
.box {  
  border: 2px solid black;  
}
```

- border-width •
- border-style •
- border-color •

Margin 4 (المسافة الخارجية)

المسافة بين العنصر والعناصر اللي حواليه

```
.box {  
  margin: 20px;  
}
```

margin مش جزء من حجم العنصر

الحساب الحقيقي للحجم

مثال:

```
.box {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}
```

الحجم النهائي =

- width = 200
- padding = 40 (يمين + شمال)
- border = 10

الإجمالي = 250px

وده سبب مشاكل كثير في التصميم

الحل السحري: box-sizing

:Default

```
box-sizing: content-box;
```

الأفضل دائمًا:

```
* {
```

```
  box-sizing: border-box;
```

```
}
```

كده:

- width يشمل padding + border
- التحكم في layout يبقى أسهل

Best Practices

- ✓ استخدم `box-sizing: border-box` دائمًا
- ✓ استخدم padding بدل margin جوه الكارد
- ✓ خَلّ margin للمسافات بين العناصر
- ✓ تجنب margin السالب إلا لو فاهم هو ليه

Common Mistakes

- ❌ نسيان إن padding ييزود الحجم
- ❌ الاعتماد على margin بدل layout
- ❌ عدم توحيد box-sizing
- ❌ مشاكل غريبة بسبب margin collapsing

Task صغير

اعمل div اسمه `card`.

حدد:

- width = 300px
- padding = 20px
- border = 2px

جرب:

- `content-box`
- `border-box`

لاحظ الفرق بنفسك

What is the Box Model?

Every element on the page is seen by the browser as a **box**

This box is made of **4 layers**:

70% of layout problems are caused by a misunderstanding of the Box Model

Content

This is the actual content:

Text

Image

Video

```
.box {  
  width: 200px;  
  height: 100px;  
}
```

Width and height apply to the **content only** (by default)

2 Padding (Inner Space)

The space between the content and the border

```
.box {  
    padding: 20px;  
}
```

Or separately:

```
padding: 10px 20px;  
padding-top: 10px;  
padding-right: 20px;
```

It **increases the element's size**

3 Border

The frame around the element

```
.box {  
    border: 2px solid black;  
}
```

Properties:

- border-width
- border-style
- border-color

Margin (Outer Space)

The space between the element and other elements around it

```
.box {  
    margin: 20px;  
}
```

Margin is **not part of the element's size**

Actual Size Calculation

Example:

```
.box {  
    width: 200px;
```

```
padding: 20px;  
border: 5px solid;  
}
```

Final size =

width = 200

padding = 40 (left + right)

border = 10

Total = 250px

This causes many design problems

Magic Solution: box-sizing

Default:

```
box-sizing: content-box;
```

Always better:

```
* {  
    box-sizing: border-box;
```

}

Now:

Width **includes padding + border**

Layout control becomes much easier

Best Practices

- ✓ Always use **box-sizing: border-box**
- ✓ Use **padding** inside cards instead of margin
- ✓ Use **margin** for spacing between elements
- ✓ Avoid negative margin unless you know why

Common Mistakes

- ✗ Forgetting that padding increases size
- ✗ Relying on margin instead of proper layout
- ✗ Not unifying box-sizing
- ✗ Strange issues due to margin collapsing

Small Task

Create a `div` named `.card`

Set:

- width = 300px
- padding = 20px
- border = 2px

Try:

- content-box
- border-box

Notice the difference yourself

Colors, fonts & units

Colors, fonts & units

إزاي نتحكم في الشكل البصري للصفحة صح؟

Colors (الألوان)

طرق كتابة الألوان في CSS

1 أسماء الألوان (مش مفضلة)

```
color: red;
```

```
background: blue;
```

محدودة ✖

مش احترافية ✖

2 HEX (الأكثر استخدامًا)

```
color: #ff0000;
```

```
background-color: #1e293b;
```

• **RRGGBB#**

• كل رقم من 0 → F

أدوات مساعدة:

Color picker •

Figma •

Coolors.co •

RGB / RGBA³

```
color: rgb(255, 0, 0);
```

```
background: rgba(0, 0, 0, 0.5);
```

RGBA مهم للـ overlays والشفافية

HSI⁴ (مهم في التصميم)

```
color: hsl(220, 80%, 50%);
```

• Hue → اللون

• Saturation → تشبع

• Lightness → الإضاءة

ممتاز لتوليد palette متناسقة

Best Practices للألوان

استخدم عدد ألوان محدود ✓

خلي فيه contrast كويس (Accessibility) ✓

استخدم CSS Variables ✓

```
:root {  
  --primary: #2563eb;  
  --secondary: #0f172a;  
}
```

Fonts (الخطوط)

أهم خصائص الخط ①

```
body {  
  font-family: 'Inter', sans-serif;  
  font-size: 16px;  
  font-weight: 400;  
  line-height: 1.6;  
}
```

Font-family 2

```
font-family: Arial, Helvetica, sans-serif;
```

دائمًا حط fallback

Font-size 3

```
font-size: 16px;
```

```
font-size: 1rem;
```

لا تستخدم px لكل حاجة

Font-weight 4

```
font-weight: 400; /* regular */
```

```
font-weight: 700; /* bold */
```

Line-height 5 (مهم جدًا)

```
line-height: 1.5;
```

قراءة أسهل 

UX أحسن 

Google Fonts 

```
<link href="https://fonts.googleapis.com/css2?family=Inter&display=swap"
rel="stylesheet">
```

Units (وحدات القياس)

Absolute Units 

```
width: 300px;
```

مش responsive 

استخدمها بحذر 

Relative Units (المفضلة) 

em •

```
font-size: 1.2em;
```

بتأثر بال parent

- rem (الأفضل للخطوط)

```
font-size: 1rem;
```

مبني على font-size `html`

- %

```
width: 50%;
```

نسبة من الـ parent

- vw / vh

```
width: 100vw;
```

```
height: 100vh;
```

مفيد للـ hero sections

أفضل استخدام	Unit
borders	px
fonts	rem
layouts	%
Full screen sections	vw/vh

3 دمج الألوان + الخطوط + الوحدات

```
.card {  
  background-color: var(--primary);  
  color: white;  
  padding: 1.5rem;  
  font-size: 1rem;  
}
```

Common Mistakes

✗ استخدام px في كل حاجة

✗ خطوط صغيرة جدًا

✗ ألوان من غير contrast

✗ أكثر من font في الموقع

Task صغير

غير لون الخلفية

استخدم CSS variables

استخدم:

- rem للخط

- % للعرض

- vh للارتفاع

جرب Google Font

Ways to write colors in CSS

1 Color Names (Not Recommended)

```
color: red;  
background: blue;
```

✗ Limited

✗ Not professional

2 HEX (Most Used)

```
color: #ff0000;  
background-color: #1e293b;
```

Format: #RRGGBB

Each digit: 0 → F

Helpful Tools:

- Color picker
- Figma

- Coolors.co

3 RGB / RGBA

```
color: rgb(255, 0, 0);  
background: rgba(0, 0, 0, 0.5);
```

RGBA is important for **overlays** and transparency

4 HSL (Important in Design)

```
color: hsl(220, 80%, 50%);
```

- Hue → Color
- Saturation → Intensity
- Lightness → Brightness

Great for generating a consistent palette

Best Practices for Colors

- ✓ Use a limited number of colors

✓ Maintain good contrast (**Accessibility**)

✓ Use CSS Variables

```
:root {  
  --primary: #2563eb;  
  --secondary: #0f172a;  
}
```

Fonts

1 Important Font Properties

```
body {  
  font-family: 'Inter', sans-serif;  
  font-size: 16px;  
  font-weight: 400;  
  line-height: 1.6;  
}
```

2 Font-family

```
font-family: Arial, Helvetica, sans-serif;
```

Always set a **fallback**

3 Font-size

```
font-size: 16px;  
font-size: 1rem;
```

Avoid using **px** for everything

4 Font-weight

```
font-weight: 400; /* regular */  
font-weight: 700; /* bold */
```

5 Line-height (Very Important)

```
line-height: 1.5;
```

✓ Easier to read

✓ Better UX

6 Google Fonts

```
<link  
href="https://fonts.googleapis.com/css2?family=Inter&display=swap" rel="stylesheet">
```

Units

1 Absolute Units

```
width: 300px;
```

✗ Not responsive

✓ Use with caution

2 Relative Units (Preferred)

- em

```
font-size: 1.2em;
```

Affected by parent

- **rem (Best for fonts)**

```
font-size: 1rem;
```

Based on `html` font-size

- **%**

```
width: 50%;
```

Percentage of parent

- **vw / vh**

```
width: 100vw;  
height: 100vh;
```

Useful for hero sections

Unit	Best Use
px	borders
rem	fonts
%	layouts
vw/vh	Full screen sections

3 Combining Colors + Fonts + Units

```
.card {
  background-color: var(--primary);
  color: white;
  padding: 1.5rem;
  font-size: 1rem;
}
```

Common Mistakes

✗ Using `px` everywhere

✗ Fonts too small

✗ Colors without contrast

✗ Using more than one font on the site

Small Task

Change the background color

Use **CSS variables**

Use:

- **rem** for font
- **%** for width
- **vh** for height

Try a **Google Font**

Display types

Display types

إزاي العناصر بتتصرف في الصفحة؟

يعني إيه `display`؟

خاصية `display` بتحدد:

- العنصر بياخد مساحة قد إيه
- بيقف جنب عناصر تانية ولا لوحده
- ينفع نتحكم في `width / height` ولا لأ

`display: block` ¹

الخصائص:

- بياخد سطر كامل
- ينزل سطر جديد بعده
- يقبل `width / height / margin / padding`

أمثلة عناصر `Block`:

`div, p, section, header, footer, article`

```
.box {  
  display: block;  
  width: 200px;  
  height: 100px;  
}
```

display: inline²

الخصائص:

- يباخذ مساحة المحتوى بس
- **ماينزلش سطر جديد**
- ❌ لا يقبل width / height
- ✔ يقبل padding أفقي فقط

أمثلة:

span, a, strong, em

```
span {  
  display: inline;  
  color: red;  
}
```

display: inline-block³

الخصائص:

- يقف جنب العناصر
- يقبل width / height
- حل وسط بين inline و block

```
.card {  
  display: inline-block;  
  width: 150px;  
  height: 100px;  
}
```

ممتاز للكروت البسيطة

display: none⁴

الخصائص:

- العنصر يختفي تمامًا
- مش يباخذ أي مساحة

```
.modal {  
  display: none;  
}
```

فرق مهم:

- `display: none` → العنصر مش موجود
- `visibility: hidden` → موجود بس مخفي

`display: flex` 5

```
.container {  
  display: flex;  
}
```

بيخلى العنصر Container مرن
بيسهّل ترتيب العناصر أفقي ورأسي

`display: grid` 6

```
display: grid
```

تقسيم الصفحة لشبكة

مناسب لـ layouts الكبيرة

الاستخدام	width/height	سطر جديد	Type
Layout	✓	✓	block
نصوص	✗	✗	inline
cards	✓	✗	inline-block
إخفاء	✗	✗	none
Layout	✓	✓	flex
Page Layout	✓	✓	grid

Common Mistakes

- ✗ محاولة تغيير width لعنصر inline
- ✗ استخدام inline-block بدل flex
- ✗ إخفاء عنصر بـ visibility بدل display
- ✗ نسيان إن block بينزل سطر جديد

Task صغير

- اعمل 3 عناصر:

- واحد block

- واحد inline

- واحد inline-block

- جرّب:

- تغيير العرض والطول

- تشوف الفرق بعينك

What is **display**?

The **display** property determines:

- How much space an element takes
- Whether it sits next to other elements or alone
- Whether you can control width / height

1 **display: block**

Properties:

- Takes the full line
- Moves to a new line after it
- Accepts width / height / margin / padding

Examples of block elements:

`div, p, section, header, footer, article`

```
.box {  
  display: block;  
  width: 200px;  
  height: 100px;  
}
```

2 display: inline

Properties:

- Takes only the content's space
- Does not move to a new line
- ✗ Does **not** accept width / height
- ✓ Accepts horizontal padding only

Examples: `span`, `a`, `strong`, `em`

```
span {  
  display: inline;  
  color: red;  
}
```

3 display: inline-block

Properties:

- Sits next to other elements

- Accepts width / height
- Middle ground between inline and block

```
.card {  
  display: inline-block;  
  width: 150px;  
  height: 100px;  
}
```

Great for simple cards

4 **display: none**

Properties:

- Element disappears completely
- Takes no space

```
.modal {  
  display: none;  
}
```

Important difference:

- `display: none` → element **does not exist**
- `visibility: hidden` → element **exists but is hidden**

5 `display: flex`

```
.container {  
  display: flex;  
}
```

- Makes the container **flexible**
- Makes arranging elements **horizontal and vertical** easier

6 `display: grid`

```
display: grid
```

- Divides the page into a grid
- Suitable for **large layouts**

Type	New Line	width/height	Usage
block	✓	✓	Layout
inline	✗	✗	Text
inline-block	✗	✓	Cards
none	✗	✗	Hiding
flex	✓	✓	Layout
grid	✓	✓	Page Layout

Common Mistakes

- ✗ Trying to change width for an inline element
- ✗ Using inline-block instead of flex
- ✗ Hiding an element with visibility instead of display
- ✗ Forgetting that block moves to a new line

Small Task

Create 3 elements:

- One block
- One inline
- One inline-block

Try:

- Changing width and height
- See the difference with your own eyes

Positioning

Positioning

إزاي نتحكم في مكان العنصر بالضبط؟

يعني إيه Position؟

خاصية `position` بتحدد:

- العنصر يتحرك بالنسبة لإيه؟
- ينفع نستخدم `left / right / bottom / top` ولا لأ؟

أنواع الـ Position

1 `position: static` (الافتراضي)

```
.box {  
  position: static;  
}
```

- الوضع الطبيعي
 -  لا يتأثر بـ `top / left`
 - يمشي مع الـ `document flow`
- 90% من العناصر `static` من غير ما تحس

position: relative²

```
.box {  
  position: relative;  
  top: 20px;  
  left: 10px;  
}
```

الخصائص:

- يتحرك نسبةً لمكانه الأصلي
- مكانه الأصلي يفضل محجوز
- أساس مهم لـ absolute

استخدام شائع جدًا: 

- تحريك بسيط
- absolute لـ Parent

position: absolute³

```
.child {  
  position: absolute;  
  top: 0;  
  right: 0;  
}
```

الخصائص:

- يطلع برا ال document flow
- يتحرك نسبةً لأقرب parent:
- `position: relative`
- `absolute / fixed`

لو مفيش parent → يتحرك بالنسبة لـ body

`position: fixed` 4

```
.navbar {  
  position: fixed;  
  top: 0;  
  width: 100%;  
}
```

الخصائص:

- ثابت في الشاشة
- مش بيتحرك مع ال scroll

استخدامات:

- Navbar
- WhatsApp button

Scroll to top •

position: sticky⁵

```
.header {  
  position: sticky;  
  top: 0;  
}
```

الخصائص:

- بين fixed و relative
- يثبت عند نقطة معينة

ممتاز للعناوين والجداول

Position	يبتحرك بالنسبة لـ؟	يأخذ مساحة؟
static	الطبيعي	✓
relative	نفسه	✓
absolute	أقرب parent	✗

✗	الشاشة	fixed
✓	Scroll position	sticky

6 z-index (الطبقات)

```
.modal {
  position: fixed;
  z-index: 1000;
}
```

يشتغل فقط مع:

- relative
- absolute
- fixed
- sticky

رقم أكبر = فوق

Common Mistakes

استخدام absolute من غير parent ❌

نسيان position قبل z-index ❌

استخدام fixed بدل sticky ❌

الLayout كامل بال position ❌

Task صغير

اعمل card

حط badge فوق يمين الكارد

Navbar ثابت فوق

زرار fixed تحت الشاشة

What is **Position**?

The `position` property determines:

- Relative to what the element moves
- Whether you can use top / right / bottom / left

Types of Position

1 `position: static (default)`

```
.box {  
  position: static;  
}
```

- Normal position
 - ✗ Not affected by top / left
- Moves with the **document flow**
- 90% of elements are static without you noticing

2 `position: relative`

```
.box {
```

```
position: relative;
top: 20px;
left: 10px;
}
```

Properties:

- Moves relative to its original position
- Original space **remains reserved**
- Important base for **absolute**

Common uses:

- Small adjustments
- Parent for absolute elements

3 position: absolute

```
.child {
position: absolute;
top: 0;
right: 0;
```

```
}
```

Properties:

- Removed from the **document flow**
- Moves relative to the nearest parent with:
`position: relative`
or `absolute / fixed`
- If no parent → moves relative to **body**

4 `position: fixed`

```
.navbar {  
  position: fixed;  
  top: 0;  
  width: 100%;  
}
```

Properties:

- Fixed on the screen
- Does not move with scroll

Uses:

- Navbar
- WhatsApp button
- Scroll to top

5 position: sticky

```
.header {  
  position: sticky;  
  top: 0;  
}
```

Properties:

- Between relative and fixed
- Sticks at a certain point

- Great for **headings and tables**

Position	Moves Relative To?	Takes Space?
static	Normal	✓
relative	Itself	✓
absolute	Nearest parent	✗
fixed	Screen	✗
sticky	Scroll position	✓

6 z-index (Layers)

```
.modal {  
  position: fixed;  
  z-index: 1000;
```

}

Works only with:

- relative
- absolute
- fixed
- sticky

Bigger number = on top

Common Mistakes

- ✗ Using absolute without a parent
- ✗ Forgetting position before z-index
- ✗ Using fixed instead of sticky
- ✗ Building the whole layout with position

Small Task

- Create a card
- Put a badge on the top-right of the card

- Navbar fixed at the top
- Button fixed at the bottom of the screen

Z-index & stacking context

Z-index & stacking context

مين فوق الثاني؟

خلينا نبدأ بالمشكلة الشهيرة:

“أنا مدي 9999 z-index

ليه العنصر لسه مستخبي؟!”

الإجابة غالبًا: Stacking Context

يعني إيه z-index؟

z-index بيحدد مين فوق مين على محور Z (العمق)

شرط مهم:

z-index بيشتغل بس على عناصر:

• **position: relative | absolute | fixed | sticky**

• أو **flex / grid children** في بعض الحالات

مثال بسيط

```
.box1 {  
  position: relative;  
  z-index: 1;  
}
```

```
.box2 {  
  position: relative;  
  z-index: 2;  
}
```

box2. هـيظهر فوق box1. ✓

طب يعني إيه Stacking Context؟

Stacking Context = عالم مستقل لـ z-index

أي عنصر جوّه Stacking Context:

- ماينفعش يطلع فوق عنصر خارج السياق ده
- موهما كان **z-index** بتاعه عالي

إمتى بيتكوّن Stacking Context؟

أهم الحالات:

- **position + z-index** غير **auto**

- **opacity** أقل من 1

- **transform**

- **filter**

- **perspective**

- **isolation: isolate**

- **will-change**

دي أكثر أسباب المشاكل

مثال:

```
.parent {  
  position: relative;  
  z-index: 1;  
}
```



```
.child {  
  position: absolute;  
  z-index: 9999;  
}
```

```
.modal {  
  position: fixed;  
  z-index: 2;  
}
```

❌ **child** مش هيطلع فوق **modal**.

ليه؟

لأنه محبوس جوه Stacking Context بتاع **parent**.

القاعدة الذهبية

z-index بيتقارن بس بين إخوات في نفس Stacking Context

Debugging Tips

- افتح DevTools
- دور على:
 - transform
 - opacity
 - position + z-index
- غالبًا السبب hidden هناك

Best Practices

- خلي ال modals / dropdowns في root level
- بلاش **z-index** أرقام مجنونة
- استخدم Scale:

```
z-index: 10; /* header */  
z-index: 100; /* dropdown */  
z-index: 1000; /* modal */
```

Task صغير

المطلوب:

اعمل layout فيه:

1. Header ثابت في أعلى الصفحة

2. Card في المحتوى

3. Modal يظهر فوق كل حاجة

الشروط:

- ال Header يكون فوق ال Card

- ال Modal يكون فوق:

 - Header

 - Card

- بدون استخدام **important!**

Let's start with the famous problem:

"I gave z-index: 9999 — why is my element still hidden?!"

The answer is usually: **Stacking Context**

What is z-index?

z-index determines **which element is on top** along the Z-axis (depth)

Important condition:

z-index only works on elements with:

position: relative | absolute | fixed | sticky

Or **flex/grid children** in some cases

Simple Example

```
.box1 {  
  
  position: relative;  
  
  z-index: 1;  
  
}
```

```
.box2 {  
  
  position: relative;  
  
  z-index: 2;  
  
}
```

✓ .box2 will appear **on top of** .box1

So what is a Stacking Context?

Stacking Context = an **independent world for z-index**

Any element inside a Stacking Context:

- Cannot go above an element outside this context
- No matter how high its z-index is

When does a Stacking Context form?

Most common cases:

- position + z-index not auto
- opacity less than 1

- transform
- filter
- perspective
- isolation: isolate
- will-change

These are the main causes of problems

Example:

```
.parent {  
  
    position: relative;  
  
    z-index: 1;  
  
}  
  
.child {  
  
    position: absolute;  
  
    z-index: 9999;  
  
}
```

```
.modal {  
  
  position: fixed;  
  
  z-index: 2;  
  
}
```

✗ `.child` will **not appear above** `.modal`

Why?

Because it is **trapped inside the Stacking Context** of `.parent`

Golden Rule

`z-index` only compares **siblings inside the same Stacking Context**

Debugging Tips

- Open **DevTools**
- Check for:

- transform
- opacity
- position + z-index
- Usually the element is hidden there

Best Practices

- Keep **modals / dropdowns** at the **root level**
- Avoid crazy z-index numbers
- Use a **scale**:

z-index: 10; /* header */

z-index: 100; /* dropdown */

z-index: 1000; /* modal */

Small Task

Create a layout with:

- Fixed Header at the top of the page
- Card in the content

- Modal that appears on top of everything

Conditions:

- Header should be **above the Card**
- Modal should be **above:**
 - Header
 - Card
- **Without using !important**

Flexbox

Flexbox

Layouts & Alignment

يعني إيه Flexbox؟

Flexbox = نظام لترتيب العناصر في اتجاه واحد

- Row (أفقي)
- Column (رأسي)

ممتاز لـ:

- Navbars
- Cards
- Buttons
- Centering

1 الفكرة الأساسية

```
.container {  
  display: flex;}  
}
```

- العنصر ده اسمه **Flex Container**
- العناصر اللي جواه اسمهم **Flex Items**

2) الاتجاه (Main Axis vs Cross Axis)

افتراضيًا:

```
flex-direction: row;
```

• Main Axis → أفقي

• Cross Axis → رأسي

تغيير الاتجاه:

```
flex-direction: column;
```

كل خواص الـ alignment بتعتمد على الاتجاه ده

3) ترتيب العناصر على الـ Main Axis

justify-content

```
justify-content: flex-start;
```

```
justify-content: center;
```

```
justify-content: space-between;
```

```
justify-content: space-around;
```

```
justify-content: space-evenly;
```

يشتغل على الاتجاه الأساسي

4 المحاذاة على الـ Cross Axis

align-items

```
align-items: stretch;  
align-items: center;  
align-items: flex-start;  
align-items: flex-end;
```

المحاذاة الرأسية (لو row)

5 توسيط عنصر في النص

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

أشهر استخدام في Flexbox

6 التحكم في التفاف العناصر

```
flex-wrap: wrap;
```

- العناصر تنزل سطر جديد
- مهم في responsive layouts

7 المسافات بين العناصر

```
gap: 20px;
```

بديل نظيف لـ margin hacks

8 التحكم في عنصر واحد

align-self

```
.item {  
  align-self: flex-end;  
}
```

يخالف align-items للعنصر ده بس

9 توزيع المساحة

flex-grow / shrink / basis

```
.item {  
  flex: 1;  
}
```

العناصر تنقسم بالتساوي

```
.item {  
  flex: 2;  
}
```

ياخذ مساحة أكبر

مثال عملي (Cards Layout)

```
.cards {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap;  
}
```

```
.card {  
  flex: 1;  
  min-width: 250px;  
}
```

Common Mistakes

- ✗ استخدام margin بدل gap
- ✗ خلط justify مع align
- ✗ نسيان flex-wrap
- ✗ محاولة عمل Grid بـ Flexbox

Task صغير

اعمل Navbar:

- Logo شمال
- Links في النص
- Button يمين

اعمل Cards:

- صف واحد
- تنزل responsive

What is **Flexbox**?

Flexbox = a system for arranging elements in **one direction**

- Row (horizontal)
- Column (vertical)

Great for:

- Navbars
- Cards
- Buttons
- Centering

1 The Basic Idea

```
.container {  
  display: flex;  
}
```

- This element is called **Flex Container**
- The elements inside are called **Flex Items**

2 Direction (Main Axis vs Cross Axis)

By default:

```
flex-direction: row;
```

- **Main Axis** → **horizontal**
- **Cross Axis** → **vertical**

Change direction:

```
flex-direction: column;
```

- All alignment properties depend on this direction

3 Aligning Items on the Main Axis

justify-content

```
justify-content: flex-start;
```

```
justify-content: center;
```

```
justify-content: space-between;
```

```
justify-content: space-around;
```

```
justify-content: space-evenly;
```

- Works along the **main axis**

4 Aligning Items on the Cross Axis

align-items

```
align-items: stretch;  
align-items: center;  
align-items: flex-start;  
align-items: flex-end;
```

- Vertical alignment (if row)

5 Centering an Element

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

- Most common Flexbox use

6 Controlling Wrapping

`flex-wrap: wrap;`

- Elements move to a new line
- Important for **responsive layouts**

7 Spacing Between Items

`gap: 20px;`

- Clean alternative to **margin hacks**

8 Control a Single Item

align-self

```
.item {  
  align-self: flex-end;  
}
```

- Overrides `align-items` for that specific item

9 Distributing Space

flex-grow / flex-shrink / flex-basis

```
.item {  
  flex: 1;  
}
```

- Items share space equally

```
.item {  
  flex: 2;  
}
```

- Takes more space

Practical Example (Cards Layout)

```
.cards {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap;  
}
```

```
.card {  
  flex: 1;  
  min-width: 250px;  
}
```

Common Mistakes

- ✗ Using margin instead of gap
- ✗ Mixing justify and align
- ✗ Forgetting flex-wrap
- ✗ Trying to make Grid with Flexbox

Small Task

- Create a Navbar:
 - Logo on the left
 - Links in the center
 - Button on the right
- Create Cards:
 - One row
 - Wrap responsively

CSS Grid (page layouts)

CSS Grid (page layouts)

يعني إيه CSS Grid؟

CSS Grid = نظام تخطيط ثنائي الأبعاد

- Rows (صفوف)
- Columns (أعمدة)

مناسب لـ:

- Page Layouts
- Dashboards
- Landing Pages
- Complex Sections

Flexbox → اتجاه واحد

Grid → صفوف + أعمدة

1 البداية مع Grid

```
.container {  
  display: grid;  
}
```


- العنصر ده اسمه **Grid Container**
- العناصر جواه اسمهم **Grid Items**

2 إنشاء الأعمدة والصفوف

Columns

```
grid-template-columns: 200px 1fr 1fr;
```

fr = جزء من المساحة المتاحة

Rows

```
grid-template-rows: auto 1fr auto;
```

3 الطريقة السحرية: repeat

```
grid-template-columns: repeat(3, 1fr);
```

نفس النتيجة بس أنصف

4 المسافات بين العناصر

```
gap: 20px;
```

بدل margin

5 التحكم في أماكن العناصر

تحديد موقع عنصر:

```
.item {  
  grid-column: 1 / 3;  
  grid-row: 1 / 2;  
}
```

من العمود 1 للعمود 3

Grid Areas 6

```
.container {  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
}
```

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.content { grid-area: content; }
```

```
.footer { grid-area: footer; }
```

واضح، scalable، readable

Media Queries بدون Responsive Grid ⁷

```
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

أعمدة مرنة حسب الشاشة

محاذاة العناصر داخل Grid ⁸

```
justify-items: center;
```

```
align-items: center;
```

عنصر واحد:

```
.item {
```

```
  justify-self: end;
```

```
}
```

مثال Page Layout كامل

```
.layout {
```

```
  display: grid;
```

```
  grid-template-columns: 250px 1fr;
```

```
  grid-template-rows: auto 1fr auto;
```

```
gap: 16px;  
}
```

Grid vs Flexbox

Grid	Flexbox
اتجاهين	اتجاه واحد
Page Layout	Component
Dashboards	Nav / Card

Common Mistakes

✗ استخدام Grid لكل حاجة صغيرة

✗ نسيان fr

✗ الـ Layout معقد بدون areas

✗ خلط Grid مع Position

Task صغير

اعمل صفحة فيها:

Header •

Sidebar •

Content •

Footer •

استخدم:

grid-template-areas •

gap •

auto-fit •

What is **CSS Grid**?

CSS Grid = **two-dimensional layout system**

- Rows
- Columns

Great for:

- Page Layouts
- Dashboards
- Landing Pages
- Complex Sections

Flexbox → **one direction**

Grid → **rows + columns**

1 Getting Started with Grid

```
.container {  
  display: grid;  
}
```

- This element is called **Grid Container**
- The elements inside are called **Grid Items**

2 Creating Columns and Rows

Columns

```
grid-template-columns: 200px 1fr 1fr;
```

- **fr** = fraction of available space

Rows

```
grid-template-rows: auto 1fr auto;
```

3 The Magic Way: repeat

```
grid-template-columns: repeat(3, 1fr);
```

- Same result but cleaner

4 Spacing Between Items

gap: 20px;

- Instead of margin

5 Controlling Item Placement

Place an item:

```
.item {  
    grid-column: 1 / 3;  
    grid-row: 1 / 2;  
}
```

- From column 1 to column 3

6 Grid Areas

```
.container {  
    grid-template-areas:  
        "header header"  
        "sidebar content"  
        "footer footer";  
}
```



```
}
```

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.content { grid-area: content; }  
.footer { grid-area: footer; }
```

- Clear, readable, scalable

7 Responsive Grid without Media Queries

```
grid-template-columns: repeat(auto-fit,  
minmax(250px, 1fr));
```

- Flexible columns depending on screen size

8 Aligning Items Inside Grid

```
justify-items: center;  
align-items: center;
```

- Single item:

```
.item {  
    justify-self: end;  
}
```

Example: Full Page Layout

```
.layout {  
    display: grid;  
    grid-template-columns: 250px 1fr;  
    grid-template-rows: auto 1fr auto;  
    gap: 16px;  
}
```

Grid vs Flexbox

Flexbox	Grid
One direction	Two directions
Component	Page Layout
Nav / Card	Dashboards

Common Mistakes

- ✗ Using Grid for every small thing
- ✗ Forgetting `fr`
- ✗ Layout becomes complex without areas
- ✗ Mixing Grid with Position

Small Task

Create a page with:

- Header
- Sidebar
- Content
- Footer

Use:

- `grid-template-areas`
- `gap`
- `auto-fit`

Responsive Design Principles

Responsive Design Principles

إزاي الموقع يشتغل صح على أي شاشة؟

يعني إيه Responsive Design؟

يعني:

- نفس الموقع
- نفس الكود
- شكل مختلف حسب حجم الشاشة

موبايل

لابتوب

ديسكتوب

Mobile First Principle 1

ابدأ دايماً بالموبايل:

```
body {  
  font-size: 14px;  
}
```

وبعدين كبر للشاشات الأكبر

- أسهل
- أداء أحسن
- تفكير سوقي صح

Flexible Layouts 2

استخدم وحدات مرنة

- px •
- % •
- rem •
- fr •

```
.container {  
  width: 90%;  
}
```

Flexible Images 3

```
img {  
  max-width: 100%;
```

```
height: auto;  
}
```

تمنع كسر التصميم

Media Queries 4

```
@media (min-width: 768px) {  
  .container {  
    display: flex;  
  }  
}
```

غير التصميم حسب حجم الشاشة

Content First 5

ركّز على:

- النص
- المحتوى
- الأزرار

مش الديكور

Touch Friendly UI 6

- زرار كبير
 - مسافات واضحة
 - ما تعتمدش على hover
- الموبايل = touch

Hide / Show Elements 7

- تمسح محتوى
- تغيير طريقة عرضه

```
.sidebar {  
  display: none;  
}
```

```
@media (min-width: 1024px) {  
  .sidebar {  
    display: block;  
  }  
}
```

Avoid Fixed Heights 8

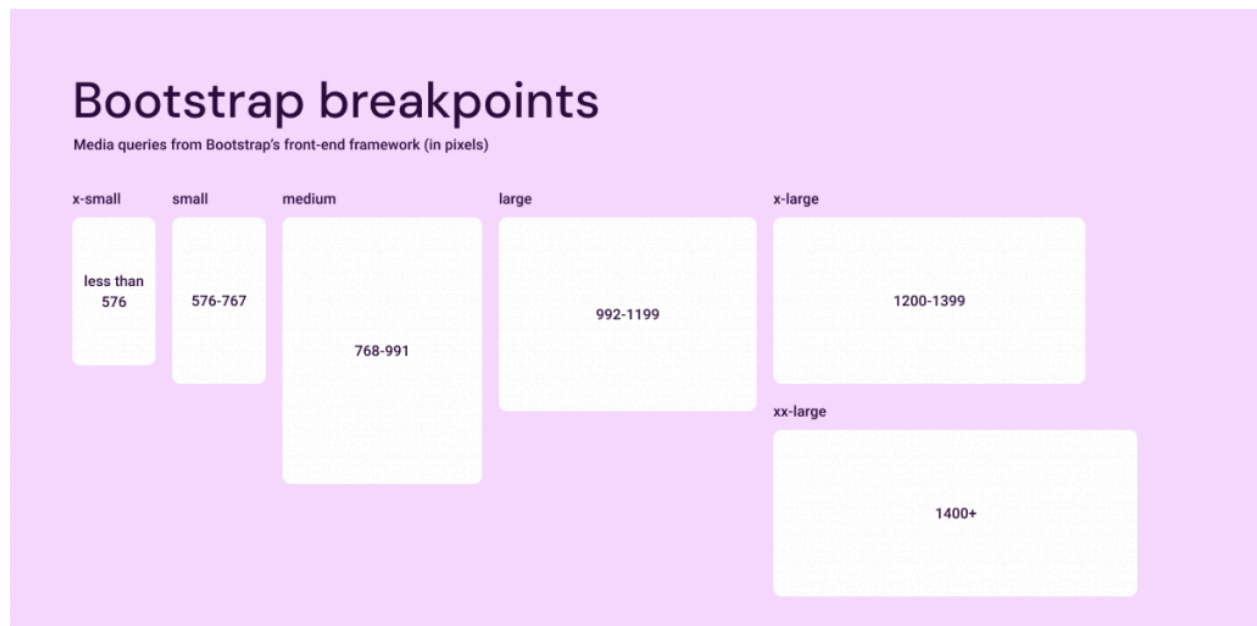
❌ مش صح

```
height: 300px;
```

✅ صح

```
min-height: 300px;
```

المحتوى مش ثابت



مثال عملي

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr;  
}
```

```
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

Common Mistakes

Desktop-first ❌

الـ px في كل حاجة ❌

نسيان الصور ❌

تصميم موبايل متأخر ❌

Task صغير

اعمل صفحة:

- كارد واحد في الموبايل
- 3 كروت في الديسكتوب

استخدم:

• Grid

Media queries •

rem •

What is **Responsive Design**?

It means:

- Same website
- Same code
- Different layout depending on **screen size**
 - Mobile
 - Laptop
 - Desktop

1 **Mobile First Principle**

Always start with mobile:

```
body {  
    font-size: 14px;  
}
```

- Then scale up for larger screens
- Easier
- Better performance

- Smart market thinking

2 Flexible Layouts

Use **flexible units**:

- px
- %
- rem
- fr

```
.container {  
  width: 90%;  
}
```

3 Flexible Images

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

- Prevents breaking the layout

4 Media Queries

```
@media (min-width: 768px) {  
  .container {  
    display: flex;  
  }  
}
```

- Change layout depending on screen size

5 Content First

Focus on:

- Text
- Content
- Buttons

Not decoration

6 Touch Friendly UI

- Big buttons
- Clear spacing
- Don't rely on hover
- Mobile = touch

7 Hide / Show Elements Properly

- Remove content
- Change how it is displayed

```
.sidebar {  
  display: none;  
}
```

```
@media (min-width: 1024px) {  
  .sidebar {  
    display: block;  
  }  
}
```


8 Avoid Fixed Heights

✗ Wrong:

```
height: 300px;
```

✓ Correct:

```
min-height: 300px;
```

- Content is not fixed

Practical Example

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr;  
}
```

```
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: repeat(3, 1fr);  
  }
```

}

Common Mistakes

- ✗ Desktop-first
- ✗ Using px for everything
- ✗ Forgetting images
- ✗ Late mobile design

Small Task

Create a page:

- One card on mobile
- Three cards on desktop

Use:

- Grid
- Media queries
- rem

Media Queries

Media Queries

تحكم في تصميمك حسب حجم الشاشة

1 يعني إيه Media Query؟

Media Query = شرط تطبيق CSS مختلفة حسب حجم الشاشة أو الجهاز.

```
@media (min-width: 768px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

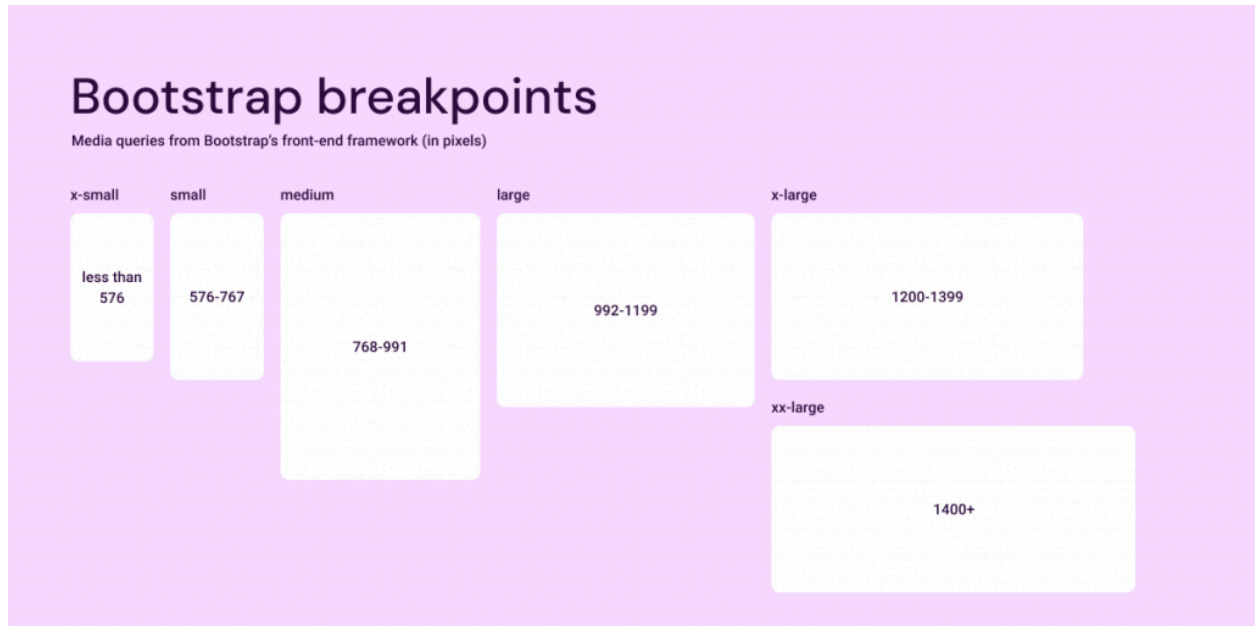
معناه: لو الشاشة 768px أو أكبر، خلي الخلفية أزرق فاتح

2 Syntax الأساسي

```
@media (condition) {  
  /* هنا CSS */  
}
```

Conditions شائعة

- `min-width` → من حجم معين وطالع
- `max-width` → لحد حجم معين
- `orientation: landscape` → أفقي
- `orientation: portrait` → عمودي



Examples 4

Example 1: تغيير layout

```
.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
```

```
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

شاشة صغيرة → 1 كارد ✓

شاشة كبيرة → 3 كروت ✓

Example 2: تغيير النصوص

```
h1 {  
  font-size: 1.5rem;  
}  
  
@media (min-width: 1024px) {  
  h1 {  
    font-size: 2.5rem;  
  }  
}
```

Example 3: Show / Hide عناصر

```
.sidebar {  
  display: none;  
}  
  
@media (min-width: 1024px) {
```

```
.sidebar {  
  display: block;  
}  
}
```

Combining Conditions 5

```
@media (min-width: 768px) and (max-width:  
1023px) {  
  body {  
    background-color: lightgreen;  
  }  
}
```

شرط مزدوج = احسن تحكم

Common Mistakes

نسيان (units (px/rem ❌

Mobile-first بدل Desktop-first ❌

تطبيق نفس ال CSS على كل الأجهزة ❌

الإفراط في breakpoints ❌

Task صغير

- اعمل صفحة فيها:

- Navbar responsive
- كارد واحد في الموبايل
- 3 كروت في الديسكتوب

- استخدم:

- min-width
- max-width
- grid-template-columns
- font-size responsive

1 What is a Media Query?

Media Query = a **condition to apply different CSS** depending on screen size or device.

```
@media (min-width: 768px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

- Meaning: if the screen is **768px or larger**, make the background light blue

2 Basic Syntax

```
@media (condition) {  
  /* CSS here */  
}
```

Common Conditions

- **min-width** → from a certain size and up

- `max-width` → up to a certain size
- `orientation: landscape` → horizontal
- `orientation: portrait` → vertical

Examples

Example 1: Change Layout

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 10px;  
}
```

```
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

-  Small screen → 1 card

-  Large screen → 3 cards

Example 2: Change Text Size

```
h1 {  
  font-size: 1.5rem;  
}
```

```
@media (min-width: 1024px) {  
  h1 {  
    font-size: 2.5rem;  
  }  
}
```

Example 3: Show / Hide Elements

```
.sidebar {  
  display: none;  
}
```

```
@media (min-width: 1024px) {  
  .sidebar {  
    display: block;  
  }  
}
```

5 Combining Conditions

```
@media (min-width: 768px) and (max-width:  
1023px) {  
  body {  
    background-color: lightgreen;  
  }  
}
```

- **Double condition = better control**

Common Mistakes

- ✗ Forgetting units (px/rem)
- ✗ Desktop-first instead of Mobile-first

- ✗ Applying the same CSS on all devices
- ✗ Overusing breakpoints

Small Task

Create a page with:

- Responsive Navbar
- One card on mobile
- Three cards on desktop

Use:

- `min-width`
- `max-width`
- `grid-template-columns`
- Responsive `font-size`

Pseudo-classes & pseudo-elements

Pseudo-classes & pseudo-elements

الفرق بينهم في جملة واحدة

• **Pseudo-class** حالة (State)

• **Pseudo-element** جزء (Part)

أولاً: Pseudo-classes

بتستخدمها لما تحب تطبق ستايل في حالة معينة

أشهرهم:

بتستخدم إمتى	Pseudo-class
لما الماوس ييجي عليه	<code>hover:</code>
وقت الكليك	<code>active:</code>
لما العنصر يتحدد	<code>focus:</code>
لينك اتفتح قبل كده	<code>visited:</code>

أول عنصر	<code>first-child:</code>
آخر عنصر	<code>last-child:</code>
عنصر معين	<code>()nth-child:</code>

مثال 1 Hover

```
button:hover {
  background-color: black;
  color: white;
}
```

مثال 2 Focus (مهم جدًا لـ Accessibility)

```
input:focus {
  outline: 2px solid blue;
}
```

مثال 3 nth-child

```
li:nth-child(2) {
  color: red;
}
```


ثانيًا: Pseudo-elements

بتستخدمها لما تحب تتحكم في جزء من العنصر

أشهرهم:

وظيفته	Pseudo-element
قبل المحتوى	<code>before::</code>
بعد المحتوى	<code>after::</code>
أول حرف	<code>first-letter::</code>
أول سطر	<code>first-line::</code>
النص المحدد	<code>selection::</code>

لازم `content` مع `before::` و `after::`

مثال 4 before / after

```
.card::before {  
  content: "🔥";  
  margin-right: 5px;  
}
```

مثال 5 Line Decoration

```
p::first-letter {  
  font-size: 32px;  
}
```

مثال 6 Selection

```
::selection {  
  background-color: yellow;  
}
```

أخطاء شائعة

- ❌ نسيان `content`
- ❌ استخدام `:` بدل `::` (رغم إنه يشتغل أحياناً)
- ❌ استخدام pseudo-elements في عناصر مش بتقبل محتوى

نصائح شغل حقيقية

- استخدم pseudo-elements بدل عناصر زيادة في HTML
- ممتازة ل:
 - icons
 - overlays

decorative elements ○

• خليك واعى بالـ Accessibility

Task صغير

المطلوب:

اعمل **Button** بالشروط دي:

1 عند **hover**

• اللون يتغير

• يظهر خط تحت الزر باستخدام **after::**

2 عند **focus**

• يظهر border واضح (Accessibility)

3 استخدم:

• Pseudo-class

• Pseudo-element

من غير إضافة أي عنصر HTML جديد

Difference in one sentence

Pseudo-class → a **State**

Pseudo-element → a **Part**

1 Pseudo-classes

Use them when you want to apply a style in a **specific state**.

Most common:

Pseudo-class	When to use
:hover	When the mouse is over it
:active	When clicking
:focus	When the element is focused
:visited	Link has been visited

:first-child	First element
:last-child	Last element
:nth-child()	Specific element

Example 1 Hover

```
button:hover {  
    background-color: black;  
    color: white;  
}
```

Example 2 Focus (important for Accessibility)

```
input:focus {  
    outline: 2px solid blue;  
}
```

Example 3 nth-child

```
li:nth-child(2) {  
    color: red;  
}
```

2 Pseudo-elements

Use them when you want to **style a part of an element**.

Most common:

Pseudo-element	Function
::before	Before content
::after	After content
::first-letter	First letter
::first-line	First line

::selection	Selected text
-------------	---------------

- **Note:** `content` is required with `::before` and `::after`

Example 4 before / after

```
.card::before {  
  content: "🔥";  
  margin-right: 5px;  
}
```

Example 5 Line Decoration

```
p::first-letter {  
  font-size: 32px;  
}
```

Example 6 Selection

```
::selection {  
  background-color: yellow;
```

}

Common Mistakes

- ✗ Forgetting **content**
- ✗ Using `:` instead of `::` (though sometimes it works)
- ✗ Using pseudo-elements on elements that don't accept content

Real Frontend Tips

- Use pseudo-elements instead of adding extra HTML elements
- Great for:
 - icons
 - overlays
 - decorative elements
- Be aware of **Accessibility**

Small Task

Create a **Button** with these requirements:

1 On **hover**:

- Change color

- Show underline using `::after`

2 On **focus**:

- Show clear border (**Accessibility**)

3 Use:

- **Pseudo-class**
- **Pseudo-element**
- Without adding any new HTML element

Overflow & object-fit

Overflow & object-fit

أولاً: Overflow

يعني إيه Overflow؟

يحصل لما المحتوى يكون أكبر من المساحة اللي واخدها العنصر.

CSS لازم تقرر:

نعمل إيه في الزيادة دي؟

قيم overflow

القيمة	بتعمل إيه
visible	(default) المحتوى يطلع برّه
hidden	يختفي
scroll	scrollbar دايمًا
auto	scrollbar عند الحاجة

مثال 1

```
.box {  
  width: 200px;  
  height: 100px;  
  overflow: hidden;  
}
```

أي محتوى زيادة مش هييان

مثال 2 Scroll

```
.box {  
  height: 100px;  
  overflow-y: auto;  
}
```

مفيد جدًا في:

- Chat
- Dropdown
- Sidebar

ملاحظات مهمة

- `overflow: hidden`
- ✗ ييقتص المحتوى
- ✗ ممكن يكسر shadow أو dropdown
- `overflow` بيكون **Block Formatting Context**
- بيأثر على `position: sticky` أحياناً

ثانيًا: Object-fit

بتستخدم إمتى؟

مع:

- `<img>`
- `<video>`

لما تحب تتحكم:

الصورة تملأ المكان إزاي من غير ما تتشوّه؟

قيم object-fit

السلوك	القيمة
يملأ (ممكّن يتشوّه)	fill
يظهر كله + فراغ	contain
يملأ من غير تشويه (أشهرهم)	cover
الحجم الطبيعي	none
أصغر اختيار	scale-down

مثال 3 صورة كارت

```
.card img {  
  width: 100%;  
  height: 200px;  
  object-fit: cover;  
}
```

ده أكثر استخدام شائع

مثال 4 Avatar

```
.avatar img {  
  width: 50px;  
  height: 50px;  
  object-fit: cover;  
  border-radius: 50%;  
}
```

Object-position

بتحدد الجزء اللي يتقص من الصورة:

```
img {  
  object-fit: cover;  
  object-position: top;  
}
```

مقارنة سريعة

المشكلة	الحل
محتوى طالع برّه	overflow
صورة مش مضبوطة	object-fit
قص النص	overflow + text-overflow

object-fit: cover	كروت ثابتة
-------------------	------------

Bug شائع

❌ استخدام background-image بدل

❌ نسيان تحديد width / height

❌ قص محتوى مهم بـ overflow: hidden

Task صغير

المطلوب:

اعمل Card فيها:

1. صورة ثابتة الارتفاع

2. نص طويل

3. زرار تحت

الشروط:

● الصورة:

○ تملأ الكارت من غير تشويه

● النص:

○ لو طويل يتقص أو يعمل scroll

● الكارت:

○ شكله ثابت مهما طول المحتوى

1 Overflow

What is Overflow?

Happens when content is **bigger than the element's space**.

CSS needs to decide: **what to do with the extra content?**

Overflow values:

Value	What it does
visible	(default) content spills out
hidden	content is hidden
scroll	always show scrollbar
auto	scrollbar when needed

Example 1

```
.box {
```

```
width: 200px;
height: 100px;
overflow: hidden;
}
```

- Any extra content **won't be visible**

Example 2 Scroll

```
.box {
  height: 100px;
  overflow-y: auto;
}
```

- Very useful for:
 - Chat
 - Dropdown
 - Sidebar

Important Notes

- `overflow: hidden`
 - ✗ Cuts content
 - ✗ Might break shadow or dropdown
- `overflow` creates a **Block Formatting Context**
- Can affect `position: sticky` sometimes

2 Object-fit

When to use?

With: `<img>` / `<video>`

When you want to **control how the image fills the space without distortion**

Object-fit values:

Value	Behavior
fill	fills (can distort)

contain	show all + empty space
cover	fills without distortion (most common)
none	original size
scale-down	smaller option

Example 3 Card Image

```
.card img {  
  width: 100%;  
  height: 200px;  
  object-fit: cover;  
}
```

- Most common use

Example 4 Avatar

```
.avatar img {  
  width: 50px;  
  height: 50px;  
  object-fit: cover;  
  border-radius: 50%;  
}
```

Object-position

- Controls which part of the image is cropped:

```
img {  
  object-fit: cover;  
  object-position: top;  
}
```

Quick Comparison:

Problem	Solution
Content spilling out	overflow
Image not fitting	object-fit
Text cutting off	overflow + text-overflow
Fixed height cards	object-fit: cover

Common Bugs

- ✗ Using `background-image` instead of `<img>`
- ✗ Forgetting to set `width / height`
- ✗ Cutting important content with `overflow: hidden`

Small Task

Create a **Card** with:

- **Fixed height image**
- **Long text**
- **Button at the bottom**

Requirements:

- **Image:** fills the card without distortion
- **Text:** if long, it should **cut off** or **scroll**
- **Card:** stays the same size regardless of content length

CSS Variables

CSS Variables (Custom Properties)

التحكم في القيم بشكل مركزي وسهل

1 يعني إيه CSS Variable؟

- زي "ثوابت" في CSS
- ممكن تخزين ألوان، خطوط، أحجام، أي قيمة CSS
- سهولة التغيير من مكان واحد

```
:root {  
  --main-color: #3498db;  
  --secondary-color: #2ecc71;  
  --font-size: 16px;  
}
```

root: → يحدد أن المتغير متاح لكل الصفحة

2 استخدام المتغير

```
body {  
  background-color: var(--main-color);  
  color: var(--secondary-color);  
  font-size: var(--font-size);  
}
```

✓ سهل التعديل

✓ يقلل تكرار الكود

3 المزايا

1 التغيير المركزي

• غير قيمة المتغير في **root** → كل العناصر تتغير

2 **Themes** (اللي هي ألوان الموقع)

```
:root {  
  --bg-color: #fff;  
  --text-color: #333;  
}
```

```
body.dark-theme {
```

```
--bg-color: #333;  
--text-color: #fff;  
}
```

```
body {  
  background-color: var(--bg-color);  
  color: var(--text-color);  
}
```

3 التعامل مع القيم الديناميكية

• calc, viewport أو حتى متغيرات ثانية

```
:root {  
  --header-height: 60px;  
}
```

```
header {  
  height: var(--header-height);  
}
```

4 Scope المتغيرات

- متغير معرف في **root** → عالمي
- متغير معرف داخل class → محلي فقط

```
.container {  
  --padding: 20px;  
}
```

```
.inner {  
  padding: var(--padding);  
}
```

5 Fallback Values

لو المتغير مش موجود:

```
color: var(--main-color, red);
```

- لو **main-color--** مش معرف → يستخدم اللون الأحمر

Common Mistakes

❌ نسيان -- في اسم المتغير

❌ استخدام المتغير قبل تعريفه

❌ إعادة تعريف نفس المتغير بطريقة غير منظمة

Task صغير

اعمل Variables للـ:

• Primary color

• Secondary color

• Font size

استخدمهم في:

• Navbar

• Buttons

• Paragraphs

جرب تغيير اللون في مكان واحد → كل العناصر تتغير

1 What is a CSS Variable?

Like “**constants**” in CSS.

Can store colors, fonts, sizes, or any CSS value.

Easy to change from one place.

```
:root {  
  --main-color: #3498db;  
  --secondary-color: #2ecc71;  
  --font-size: 16px;  
}
```

- `:root` → makes the variable **available globally**

2 Using the Variable

```
body {  
  background-color: var(--main-color);  
  color: var(--secondary-color);  
  font-size: var(--font-size);  
}
```

- ✓ Easy to update
- ✓ Reduces code repetition

3 Benefits

1 Centralized changes

- Change the variable in `:root` → all elements update

2 Themes (site colors)

```
:root {  
  --bg-color: #fff;  
  --text-color: #333;  
}
```

```
body.dark-theme {  
  --bg-color: #333;  
  --text-color: #fff;  
}
```

```
body {  
  background-color: var(--bg-color);  
  color: var(--text-color);  
}
```

3 Dynamic values

- Can use `calc()`, viewport units, or even other variables

```
:root {
```



```
--header-height: 60px;
}

header {
  height: var(--header-height);
}
```

4 Variable Scope

- Variable in `:root` → global
- Variable inside a class → local only

```
.container {
  --padding: 20px;
}

.inner {
  padding: var(--padding);
}
```

5 Fallback Values

- If variable doesn't exist:

```
color: var(--main-color, red);
```

- If `--main-color` is not defined → uses red

Common Mistakes

- ✗ Forgetting `--` in the variable name
- ✗ Using the variable **before defining it**
- ✗ Redefining the same variable in a messy way

Small Task

Create **Variables** for:

- Primary color
- Secondary color
- Font size

Use them in:

- Navbar
- Buttons
- Paragraphs

Try changing the color in one place → all elements update

Transitions & Animations

CSS Transitions & Animations

إزاي نخلي العناصر تتحرك أو تتغير بشكل سلس

Transitions 1

تغيير سلس للقيم لما يحصل event معين

مثال: hover على زر

```
button {  
  background-color: #3498db;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  border-radius: 6px;  
  transition: background-color 0.3s ease, transform 0.3s ease;  
}
```

```
button:hover {
```

```
background-color: #2ecc71;  
transform: scale(1.1);  
}
```

شرح:

- `transition-property` → الخصائص التي ستتغير (`background-color`, `transform`)

- `transition-duration` → المدة الزمنية للتغيير (`0.3s`)

- `transition-timing-function` → نوع الحركة (`ease`)

كل تغيير يحصل على `hover` يبقى `smooth` بدون أي جمود

2 Animation Transform بسيطة

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  transition: transform 0.5s;  
}
```

```
.box:hover {  
  transform: rotate(45deg) scale(1.2);  
}
```

- `rotate()` → تدوير
- `scale()` → تكبير/تصغير
- `translate()` → تحريك

Transforms + Transitions → حركات سلسلة جدًا

Keyframes Animation 3

للحركات المعقدة والمستمرة

```
@keyframes bounce {  
  0% { transform: translateY(0); }  
  50% { transform: translateY(-20px); }  
  100% { transform: translateY(0); }  
}
```

```
.bounce-box {
```

```
width: 100px;  
height: 100px;  
background-color: orange;  
animation: bounce 1s infinite;  
}
```

شرح:

- `keyframes@` → يحدد خطوات الحركة
- `animation-name` → اسم الحركة
- `animation-duration` → المدة الزمنية
- `animation-iteration-count` → عدد مرات التكرار (`infinite` = لا نهائي)

Animation Shorthand 4

```
animation: bounce 2s ease-in-out infinite alternate;
```

- `ease-in-out` → نوع الحركة
- `alternate` → الرجوع بالعكس بعد كل دورة

Combining Transitions & Animations 5

Transitions → best for **hover / click** •

Keyframes → best for **continuous / looping animation** •

مثالي ل:

Buttons hover •

Loading spinners •

Attention grabbing elements •

Common Mistakes

تغيير خاصية غير animatable (مثل display) ❌

استخدام transition على كل شيء (all) بدون داعي ❌

durations قصيرة جدًا → حركة غير ملحوظة ❌

overuse → مشتت المستخدم ❌

Task صغير

Button hover animation 1

• تغيير لون الخلفية

- تكبير حجم الزرار

- تحريك بسيط

2>Loading box animation:

- يتحرك للأعلى والأسفل

- تكرار لا نهائي

1 Transitions

Smooth change of values when an **event happens** (like hover on a button).

```
button {  
    background-color: #3498db;  
    color: white;  
    padding: 10px 20px;  
    border: none;  
    border-radius: 6px;  
    transition: background-color 0.3s ease,  
    transform 0.3s ease;  
}
```

```
button:hover {  
    background-color: #2ecc71;  
    transform: scale(1.1);  
}
```

Explanation:

- **transition-property** → which properties will change (e.g., background-color, transform)
- **transition-duration** → how long the change takes (0.3s)
- **transition-timing-function** → type of movement (ease)

✓ Every hover change is **smooth** without jumps

2 Transform (Simple Animation)

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  transition: transform 0.5s;  
}
```

```
.box:hover {  
  transform: rotate(45deg) scale(1.2);  
}
```

Functions:

- `rotate()` → rotate element
- `scale()` → enlarge/shrink
- `translate()` → move

Transitions + Transforms → smooth and nice animations

3 Keyframes Animation

For **complex or continuous movements**

```
@keyframes bounce {  
  0%    { transform: translateY(0); }  
  50%   { transform: translateY(-20px); }  
  100%  { transform: translateY(0); }  
}
```

```
.bounce-box {  
  width: 100px;  
  height: 100px;  
  background-color: orange;
```

```
animation: bounce 1s infinite;  
}
```

Explanation:

- `@keyframes` → defines the steps of the animation
- `animation-name` → name of the animation
- `animation-duration` → duration
- `animation-iteration-count` → number of repetitions
(`infinite` = forever)

4 Animation Shorthand

```
animation: bounce 2s ease-in-out infinite  
alternate;
```

- `ease-in-out` → type of easing
- `alternate` → reverse direction every cycle

5 Combining Transitions & Animations

- **Transitions** → best for hover / click events

- **Keyframes** → best for continuous / looping animations

✓ Perfect for:

- Buttons hover
- Loading spinners
- Attention grabbing elements

Common Mistakes

- ✗ Animating non-animatable properties (like `display`)
- ✗ Using `transition: all` without reason
- ✗ Very short durations → animation not noticeable
- ✗ Overuse → distracts users

Small Task

1 Button hover animation:

- Change background color
- Enlarge button
- Small movement

2 Loading box animation:

- Move up and down
- Repeat infinitely

Transforms Animations

CSS Transforms Animations

تحريك العناصر بطريقة ديناميكية وسلسة

1 يعني إيه Transform؟

Transform = تغيير شكل العنصر دون التأثير على مكانه في DOM

- تحريك
- تدوير
- تكبير / تصغير
- ميل / skew

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  transition: transform 0.5s ease;  
}
```

```
.box:hover {  
  transform: translateX(50px) rotate(20deg)  
  scale(1.2);  
}
```

شرح:

- `translateX/Y` → تحريك العنصر على المحور الأفقي/الرأسي
- `rotate()` → تدوير العنصر
- `scale()` → تكبير/تصغير
- `skew()` → ميل العنصر

يمكن تجمع Transformات متعددة معًا

2 Transform Functions شائعة

Function	معنى	مثال
<code>translateX(50px)</code>	تحريك أفقي	<code>transform: translateX(50px)</code>
<code>translateY(30px)</code>	تحريك رأسي	<code>transform: translateY(30px)</code>
<code>rotate(45deg)</code>	تدوير	<code>transform: rotate(45deg)</code>
<code>scale(1.5)</code>	تكبير	<code>transform: scale(1.5)</code>
<code>skew(20deg, 10deg)</code>	ميل	<code>transform: skew(20deg, 10deg)</code>

Combining Transforms 3

```
transform: translateX(50px) rotate(45deg)  
scale(1.2);
```

- كل Transform يتم ترتيبه حسب التسلسل
- ممكن يعمل حركة متكاملة سلسلة

Transform Origin 4

```
transform-origin: top left;
```

- النقطة الي حولها يتم التدوير أو التكبير
- Default → center center

Transforms + Transitions 5

```
.box {  
  transition: transform 0.4s ease-in-out;  
}
```

```
.box:hover {  
  transform: rotate(15deg) scale(1.1);  
}
```

الحركة هتبقى سلسلة عند hover

Transforms + Keyframes (Animation 6) 6 مستمرة

```
@keyframes rotate-scale {  
  0%   { transform: rotate(0deg) scale(1); }  
  50%  { transform: rotate(180deg) scale(1.5); }  
  100% { transform: rotate(360deg) scale(1); }  
}
```

```
.box {  
  animation: rotate-scale 2s infinite linear;  
}
```

- تحريك مستمر
- مثالي للـ **loaders** أو **decorative animations**

Common Mistakes

- ✗ استخدام Transform → **display: none** مش هتشتغل
- ✗ durations قصيرة جدًا → حركة غير ملحوظة
- ✗ overuse → مشيت للمستخدم

Task صغير

Task 1: Hover Effect

- زرار يتحرك يممين ويسار
- يدور شوية
- يكبر قليلاً عند المرور عليه

Task 2: Continuous Animation

- مربع يدور 360° بشكل مستمر
- يكبر ويصغر بالتناوب
- استخدام `keyframes + transform@`

1 What is Transform?

Transform = change the shape of an element **without affecting its place in the DOM**

Move

Rotate

Scale

Skew

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  transition: transform 0.5s ease;  
}
```

```
.box:hover {  
  transform: translateX(50px) rotate(20deg)  
  scale(1.2);  
}
```

Explanation:

- `translateX/Y` → move the element horizontally/vertically
- `rotate()` → rotate the element
- `scale()` → enlarge/shrink
- `skew()` → skew the element

✓ You can combine multiple transforms together

2 Common Transform Functions

Function	Meaning	Example
<code>translateX(50px)</code>	Move horizontally	<code>transform: translateX(50px)</code>
<code>translateY(30px)</code>	Move vertically	<code>transform: translateY(30px)</code>
<code>rotate(45deg)</code>	Rotate	<code>transform: rotate(45deg)</code>
<code>scale(1.5)</code>	Scale	<code>transform: scale(1.5)</code>

skew(20deg, 10deg)	Skew	transform: skew(20deg, 10deg)
-----------------------	------	----------------------------------

3 Combining Transforms

```
transform: translateX(50px) rotate(45deg)  
scale(1.2);
```

- ✓ Each transform is applied in sequence
- ✓ Can create a smooth combined movement

4 Transform Origin

```
transform-origin: top left;
```

The point around which rotation or scaling happens

Default → **center center**

5 Transforms + Transitions

```
.box {  
  transition: transform 0.4s ease-in-out;  
}
```

```
.box:hover {  
  transform: rotate(15deg) scale(1.1);  
}
```

✓ Movement is smooth on hover

6 Transforms + Keyframes (Continuous Animation)

```
@keyframes rotate-scale {  
  0%    { transform: rotate(0deg) scale(1); }  
  50%   { transform: rotate(180deg) scale(1.5); }  
  100%  { transform: rotate(360deg) scale(1); }  
}
```

```
.box {  
  animation: rotate-scale 2s infinite linear;  
}
```

- ✓ Continuous movement
- ✓ Perfect for loaders or decorative animations

Common Mistakes

- ✗ Using `display: none` → Transform will not work
- ✗ Very short durations → animation not noticeable
- ✗ Overuse → distracts the user

Small Task

Task 1: Hover Effect

- Button moves left and right
- Rotates slightly
- Slightly grows on hover

Task 2: Continuous Animation

- Box rotates 360° continuously
- Grows and shrinks alternately
- Use `@keyframes` + transform

Keyframes Animations

CSS Keyframes Animations

الحركات المعقدة والمستمرة للصفحة

1 يعني إيه Keyframes Animation؟

- Keyframes = خطوات الحركة
- بتحدد **نقاط البداية، المنتصف، والنهاية** للحركة
- بيشتغل مع **animation** property

```
@keyframes slide {  
  0% { transform: translateX(0); }  
  50% { transform: translateX(100px); }  
  100% { transform: translateX(0); }  
}
```

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;
```

```
animation: slide 2s infinite;  
}
```

Syntax 2 أساسي

```
@keyframes animation-name {  
  0% { /* بداية */ }  
  50% { /* منتصف */ }  
  100% { /* نهاية */ }  
}
```

```
.element {  
  animation-name: animation-name;  
  animation-duration: 2s;  
  animation-iteration-count: infinite;  
  animation-timing-function: ease-in-out;  
  animation-direction: alternate;  
}
```

Animation Properties مهمة للـ 3

المعنى	Property
اسم الـ keyframes	animation-name
مدة الحركة	animation-duration
عدد مرات التكرار (<i>infinite</i>)	animation-iteration-count
نوع الحركة (<i>linear, ease-in-out</i>)	animation-timing-function
تأخير بداية الحركة	animation-delay
الاتجاه (<i>normal, alternate</i>)	animation-direction

4 أمثلة شائعة

Example 1: Bounce

```
@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
}
```

```
.ball {
  width: 50px;
```

```
height: 50px;  
background-color: orange;  
border-radius: 50%;  
animation: bounce 1s infinite;  
}
```

Example 2: Rotate & Scale

```
@keyframes rotate-scale {  
  0% { transform: rotate(0deg) scale(1); }  
  50% { transform: rotate(180deg) scale(1.5); }  
  100% { transform: rotate(360deg) scale(1); }  
}
```

```
.cube {  
  width: 100px;  
  height: 100px;  
  background-color: purple;  
  animation: rotate-scale 2s infinite linear;  
}
```


Example 3: Fade In / Fade Out

```
@keyframes fade {  
  0% { opacity: 0; }  
  50% { opacity: 1; }  
  100% { opacity: 0; }  
}
```

```
.text {  
  animation: fade 3s infinite;  
}
```

Tips & Best Practices **5**

- استخدم Transform بدل top/left لتحسين الأداء
- لا تستخدم أكثر من 3-4 Animations في نفس الوقت على العنصر الواحد
- استخدم **will-change: transform**; لتحسين الأداء في المتصفحات
- Combine Transitions + Keyframes للمؤثرات التفاعلية

Common Mistakes

❌ استخدام `display` في Animation

❌ الـ durations قصيرة جدًا

❌ overuse → مشتت المستخدم

❌ تجاهل الأداء على الموبايل

Task صغير

Task 1: Hover Button Animation

- عند hover:
- تكبير حجم الزرار
- تغيير اللون
- تدوير بسيط

Task 2: Continuous Loader Animation

- دائرة تدور 360° باستمرار
- تكبير وتصغير تدريجي
- استخدام `keyframes + transform + scale@`

1 What is Keyframes Animation?

Keyframes = steps of the animation

Defines start, middle, and end points of a movement

Works with the **animation** property

```
@keyframes slide {
```

```
0% { transform: translateX(0); }
```

```
50% { transform: translateX(100px); }
```

```
100% { transform: translateX(0); }
```

```
}
```

```
.box {
```

```
width: 100px;
```

```
height: 100px;
```

```
background-color: red;
```

```
animation: slide 2s infinite;
```

```
}
```

2 Basic Syntax

```
@keyframes animation-name {
```

```
  0% { /* start */ }
```

```
  50% { /* middle */ }
```

```
  100% { /* end */ }
```

```
}
```

```
.element {
```

```
  animation-name: animation-name;
```

```
  animation-duration: 2s;
```

```
  animation-iteration-count: infinite;
```

```
  animation-timing-function: ease-in-out;
```

```
  animation-direction: alternate;
```

```
}
```

3 Important Animation Properties

Property	Meaning
animation-name	Name of the keyframes
animation-duration	Duration of the animation
animation-iteration-count	Number of repetitions (infinite)
animation-timing-function	Type of movement (linear, ease-in-out)
animation-delay	Delay before the animation starts
animation-direction	Direction (normal, alternate)

4 Common Examples

Example 1: Bounce

```
@keyframes bounce {  
  
  0%, 100% { transform: translateY(0); }  
  
  50% { transform: translateY(-20px); }  
  
}
```

```
.ball {  
  
  width: 50px;  
  
  height: 50px;  
  
  background-color: orange;  
  
  border-radius: 50%;  
  
  animation: bounce 1s infinite;  
  
}
```

Example 2: Rotate & Scale

```
@keyframes rotate-scale {
```

```
0% { transform: rotate(0deg) scale(1); }  
  
50% { transform: rotate(180deg) scale(1.5); }  
  
100% { transform: rotate(360deg) scale(1); }  
  
}
```

```
.cube {  
  
width: 100px;  
  
height: 100px;  
  
background-color: purple;  
  
animation: rotate-scale 2s infinite linear;  
  
}
```

Example 3: Fade In / Fade Out

```
@keyframes fade {  
  
0% { opacity: 0; }
```

```
50% { opacity: 1; }  
  
100% { opacity: 0; }  
  
}
```

```
.text {  
  
  animation: fade 3s infinite;  
  
}
```

5 Tips & Best Practices

- ✓ Use **transform** instead of **top/left** for better performance
- ✓ Don't use more than 3–4 animations on the same element at once
- ✓ Use **will-change: transform;** to optimize performance in browsers
- ✓ Combine Transitions + Keyframes for interactive effects

Common Mistakes

- ✗ Using `display` in animation
- ✗ Very short durations → animation not noticeable
- ✗ Overuse → distracts the user
- ✗ Ignoring performance on mobile

Small Task

Task 1: Hover Button Animation

- On hover:
 - Enlarge the button
 - Change its color
 - Slight rotation

Task 2: Continuous Loader Animation

- Circle rotates 360° continuously

- Grows and shrinks gradually
- Use @keyframes + transform + scale

Best Practices & Common Mistakes

CSS Best Practices & Common Mistakes

CSS Syntax & Selectors 1

:Best Practices

- استخدم **selectors محددة وواضحة** لتجنب التعارض
- حافظ على **تنسيق الكود** (indentation, spacing)
- استعمل **class** بدل **id** للـ styling غالبًا

:Common Mistakes

- **selectors** عامة جدًا → **div {}**
- تكرار نفس الخصائص في كذا مكان
- نسيان **;** أو **{ }** → خطأ syntax

Box Model 2

:Best Practices

- افهم **padding, border, margin** كويس

- استخدم `box-sizing: border-box` لتسهيل الحسابات
- حافظ على وحدة قياس واحدة (px / rem / em)

:Common Mistakes

- تجاهل borders عند الحساب → layout بيتخرب
- استخدام ارتفاع ثابت بدون `min-height` → مشاكل في المحتوى

Colors, Fonts & Units 3

:Best Practices

- استخدم **CSS Variables** للألوان والخطوط
- استخدم **rem/em** للخطوط والمساحات بدل px للـ responsiveness
- اختر ألوان contrast مناسبة للـ accessibility

:Common Mistakes

- كثرة الألوان بدون نظام
- استخدام px فقط → صعوبة الـ responsive
- تجاهل ألوان النص مقابل الخلفية

Display Types 4

:Best Practices

- فهم الفرق بين **block / inline / inline-block / flex / grid**
- استخدم display حسب الحاجة لتجنب التعارضات

:Common Mistakes

- استخدام **inline** بدل **inline-block** لما تحتاج حجم
- محاولة Flex داخل layout → inline element ييتخرب

Positioning 5

:Best Practices

- افهم الفرق بين **static, relative, absolute, fixed, sticky**
- استخدم absolute + relative بدل absolute وحيد في كثير من الأحيان
- حافظ على hierarchy واضحة

:Common Mistakes

- استخدام absolute لكل العناصر → مشاكل responsiveness

- تجاهل (z-index stacking context)

Flexbox 6

:Best Practices

- استخدم Flexbox لترتيب عناصر داخل container
- استخدم **justify-content & align-items** بدل margin hacks
- Combine flex + media queries لل responsive

:Common Mistakes

- استخدام float بدل flex
- عدم تحديد flex-wrap → العناصر تخرج عن ال container

CSS Grid 7

:Best Practices

- استخدم grid-template-columns/rows بوضوح
- استعمل fractional units **fr** / auto-fit / auto-fill

- responsive لا Combine Grid + Media Queries

:Common Mistakes

- استخدام grid على عنصر واحد فقط → مفيد الاستخدام
- تجاهل spacing → gap غير منظم

Responsive Design Principles 8

:Best Practices

- Mobile-first approach
- استخدام units مرنة (% ,vw, vh, rem)
- تحقق من تصميمك على كل الأجهزة

:Common Mistakes

- Desktop-first approach
- تجاهل الصور والوسائط
- breakpoints كثيرة أو غير منطقية

Media Queries 9

:Best Practices

- استخدم min-width للموبايل الأول (mobile-first)
- اجمع breakpoints حسب الحاجة
- استخدم conditions مزدوجة عند الضرورة

:Common Mistakes

- نسخ نفس CSS لكل breakpoint
- durations أو widths غير متسقة
- overuse → layout معقد

CSS Variables 10

:Best Practices

- عرف variables في root: للموقع كله
- استخدمها لتغيير themes / ألوان / خطوط
- حافظ على أسماء --primary-color → descriptive

:Common Mistakes

- نسيان -- في الاسم
- إعادة تعريف المتغيرات بدون نظام
- استخدام variables قبل تعريفها

Transitions & Transforms Animations 11

:Best Practices

- استخدم transitions لل hover / click
- استخدم transform بدل top/left لتحسين الأداء
- durations معقولة (0.2-0.5s عادة)

:Common Mistakes

- overuse → مشتت للمستخدم
- duration قصيرة جدًا → غير محسوس
- محاولة animate display → لا تعمل

Keyframes Animations 12

:Best Practices

- استخدم transform + opacity بدل top/left
- استخدم animation-direction: alternate للحركة الطبيعية
- اجعل الحركة سلسلة باستخدام timing functions

:Common Mistakes

- الكثير من الـ animations في نفس الوقت
- استخدام display أو properties غير animatable
- durations قصيرة جدًا → الحركة غير ملحوظة

1 CSS Syntax & Selectors

Best Practices:

- Use clear and specific selectors to avoid conflicts
- Keep code formatted (indentation, spacing)
- Prefer `class` over `id` for styling in most cases

Common Mistakes:

- Too general selectors → `div {}`
- Repeating the same properties in multiple places
- Forgetting `;` or `{}` → syntax error

2 Box Model

Best Practices:

- Understand padding, border, margin well

- Use `box-sizing: border-box` for easier calculations
- Keep consistent units (px / rem / em)

Common Mistakes:

- Ignoring borders when calculating → layout breaks
- Using fixed height without `min-height` → content issues

3 Colors, Fonts & Units

Best Practices:

- Use CSS Variables for colors and fonts
- Use rem/em for fonts and spacing instead of px for responsiveness
- Choose high contrast colors for accessibility

Common Mistakes:

- Too many colors without a system
- Using only px → responsive difficulties
- Ignoring text color vs background contrast

4 Display Types

Best Practices:

- Understand the difference between block / inline / inline-block / flex / grid
- Use `display` according to need to avoid conflicts

Common Mistakes:

- Using inline instead of inline-block when size is needed
- Trying Flex inside an inline element → layout breaks

5 Positioning

Best Practices:

- Understand static, relative, absolute, fixed, sticky
- Use relative + absolute instead of standalone absolute in many cases
- Keep a clear hierarchy

Common Mistakes:

- Using absolute for everything → responsiveness issues
- Ignoring stacking context (z-index)

6 Flexbox

Best Practices:

- Use Flexbox to arrange elements inside a container

- Use `justify-content` & `align-items` instead of margin hacks
- Combine flex + media queries for responsive design

Common Mistakes:

- Using float instead of flex
- Not specifying `flex-wrap` → elements overflow container

7 CSS Grid

Best Practices:

- Use `grid-template-columns/rows` clearly
- Use fractional units `fr` and `auto-fit` / `auto-fill`
- Combine Grid + Media Queries for responsive design

Common Mistakes:

- Using grid on a single element only → no real benefit
- Ignoring **gap** → irregular spacing

8 Responsive Design Principles

Best Practices:

- Mobile-first approach
- Use flexible units (% , rem, vw, vh)
- Test your design on all devices

Common Mistakes:

- Desktop-first approach
- Ignoring images and media
- Too many or illogical breakpoints

9 Media Queries

Best Practices:

- Use min-width for mobile-first approach
- Group breakpoints as needed
- Use combined conditions when necessary

Common Mistakes:

- Copying the same CSS for each breakpoint
- Inconsistent durations or widths
- Overuse → complicated layout

10 CSS Variables

Best Practices:

- Define variables in `:root` for global use

- Use them for themes / colors / fonts
- Keep descriptive names → `--primary-color`

Common Mistakes:

- Forgetting `--` in the name
- Redefining variables without a system
- Using variables before defining them

Transitions & Transform Animations

Best Practices:

- Use transitions for hover / click
- Use `transform` instead of top/left for better performance
- Keep reasonable durations (0.2–0.5s usually)

Common Mistakes:

- Overuse → distracts the user
- Very short durations → not noticeable
- Trying to animate `display` → won't work

Keyframes Animations

Best Practices:

- Use transform + opacity instead of top/left
- Use `animation-direction: alternate` for natural motion
- Make motion smooth using timing functions

Common Mistakes:

- Too many animations at the same time

- Using display or non-animatable properties
- Very short durations → animation not noticeable

Flexbox vs Grid

Flexbox vs Grid

الاثنتين أدوات Layout في CSS

لكن كل واحد له وظيفة مختلفة
وأكثر غلطة إنك تستخدمهم عشوائي.

الفكرة الأساسية

Flexbox = Layout في اتجاه واحد

Grid = Layout في اتجاهين

• Flexbox → صف أو عمود

• Grid → صفوف وأعمدة مع بعض

أولاً: Flexbox

إمتى تستخدم Flexbox؟

استخدم Flexbox لما:

- يكون عندك عناصر في صف واحد
- أو عمود واحد
- ومحتاج تتحكم في:
 - المحاذاة
 - المسافات
 - ترتيب العناصر

أمثلة مناسبة لـ Flexbox

- Navbar
- Buttons group
- Card content
- Centering element
- List items

مثال:

```
    } navbar.  
    ;display: flex  
;justify-content: space-between  
    ;align-items: center
```


{

هنا عندك اتجاه واحد (Row)

طريقة التفكير مع Flexbox

اسأل نفسك:

- العناصر ماشية في Row ولا Column؟
- مين الـ main axis؟
- محتاج Align ولا Distribute؟

ثانياً: Grid

إمتى تستخدم Grid؟

استخدم Grid لما:

- الصفحة متقسمة بوضوح
- عندك Rows و Columns

- محتاج تحكم كامل في المكان

أمثلة مناسبة لـ Grid

- Page layout
- Dashboard
- Gallery
- Cards grid
- Main website structure

مثال:

```
    } layout.  
    ;display: grid  
;grid-template-columns: 250px 1fr  
;grid-template-rows: auto 1fr auto  
    {
```

هنا بتبني هيكل صفحة كامل

طريقة التفكير مع Grid

اسأل نفسك:

- الصفحة متقسمة إزاي؟
- كل عنصر مكانه فين؟
- في علاقة بين الصفوف والأعمدة؟

مقارنة سريعة

Grid	Flexbox
اتجاهين	اتجاه واحد
Layout-based	Content-based
أقوى وأدق	سهل وسريع
مثالي للصفحة	مثالي للكومبوننت

 استخدامهم مع بعض (وده الصح غالبًا)

مش يا Flexbox يا Grid

غالبًا هتستخدم الاثنين مع بعض

مثال:

- Grid → تقسيم الصفحة
- Flexbox → ترتيب المحتوى جوه كل قسم

```
    } page.  
    ;display: grid  
;grid-template-columns: 1fr 3fr  
    {  
  
    } card.  
    ;display: flex  
;flex-direction: column  
    ;gap: 1rem  
    {
```

ده التفكير الاحترافي.

❌ أخطاء شائعة

- استخدام Grid لزرارين جنب بعض
- استخدام Flexbox لتقسيم Dashboard كامل
- حفظ properties بدون فهم

قاعدة ذهنية

اسأل نفسك سؤال واحد:

هل المشكلة في ترتيب عناصر؟

Flexbox →

ولا في تقسيم مساحة؟

Grid →

الخلاصة

- Flexbox = alignment & flow
- Grid = structure & layout
- الاثنين مكملين لبعض مش بدائل

Both Flexbox and Grid are CSS layout tools,
but **they solve different problems**.

The biggest mistake is using them randomly without understanding their purpose.

Core Concept

Flexbox = One-dimensional layout

Grid = Two-dimensional layout

- Flexbox → row **or** column
- Grid → rows **and** columns together

First: Flexbox

When should you use Flexbox?

Use Flexbox when:

- You are laying out items in a **single row or column**

- You need to control:
 - Alignment
 - Spacing
 - Order of items

Good Use Cases for Flexbox

- Navigation bars
- Button groups
- Card content
- Centering elements
- Lists and menus

Example:

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

This is a one-direction (row) layout.

How to Think with Flexbox

Ask yourself:

- Are items flowing in a row or a column?
- What is the main axis?
- Do I need alignment or distribution?

Second: Grid

When should you use Grid?

Use Grid when:

- The page has a clear structure
- You need both rows and columns
- You want full control over layout placement

Good Use Cases for Grid

- Page layouts

- Dashboards
- Image galleries
- Card grids
- Main website structure

Example:

```
.layout {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: auto 1fr auto;  
}
```

This example defines a full page structure.

How to Think with Grid

Ask yourself:

- How is the page divided?
- Where should each element live?

- What is the relationship between rows and columns?

Quick Comparison

Flexbox	Grid
One direction	Two directions
Content-based	Layout-based
Simple & flexible	Powerful & precise
Best for components	Best for page structure

Using Flexbox and Grid Together (Best Practice)

It's not **Flexbox vs Grid** —

Most real projects use **both together**

Example:

- Grid → page structure
- Flexbox → content inside sections

```
.page {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
}  
  
.card {  
  display: flex;  
  flex-direction: column;  
  gap: 1rem;  
}
```

This is professional frontend thinking.

Common Mistakes

- Using Grid for two buttons side by side
- Using Flexbox to build an entire dashboard layout
- Memorizing properties without understanding use cases

Golden Rule

Ask yourself one question:

Is the problem about aligning items?

→ Use Flexbox

Or is it about dividing space and layout structure?

→ Use Grid

Summary

- Flexbox = alignment & flow
- Grid = structure & layout
- They complement each other — not competitors

Exercise: Animated Card Layout

Exercise: Animated Card Layout

الهدف:

- استخدام (CSS Layout (Flexbox/Grid
- تطبيق Colors, Fonts & Units
- استخدام Transitions, Transforms, Keyframes Animations
- تجربة Hover effects و interactive UI

1 فكرة المشروع

- صفحة فيها **بطاقات Cards** تمثل منتجات أو مقالات أو خدمات
- كل بطاقة فيها:
 - صورة
 - عنوان
 - وصف قصير
 - زر أو رابط
- المطلوب:
 - عرض البطاقات بطريقة **grid** أو **flex responsive**
 - عند hover على البطاقة:

- تكبير البطاقة قليلاً (scale)
- تظهر معلومات إضافية (مثلاً overlay أو description)
- تغيير اللون أو الظل (box-shadow)
- إمكانية إضافة **animation مستمرة أو تأثيرات keyframes**
 - (مثل حركة صغيرة للزرار أو صورة المنتج)

HTML Structure 2

```
<div class="cards-container">
  <div class="card">
    
    <h3>Product 1</h3>
    <p>Short description about the product.</p>
    <button>Learn More</button>
  </div>
  <div class="card">
    
    <h3>Product 2</h3>
    <p>Short description about the product.</p>
    <button>Learn More</button>
  </div>
  <!-- More cards -->
</div>
```

a) Layout & Grid / Flex

```
.cards-container {  
  display: grid;  
  grid-template-columns: repeat(auto-fit,  
minmax(250px, 1fr));  
  gap: 20px;  
  padding: 20px;  
}
```

b) Card Base Styles

```
.card {  
  background-color: white;  
  border-radius: 12px;  
  overflow: hidden;  
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);  
  transition: transform 0.3s ease, box-shadow  
0.3s ease;  
}
```

c) Hover Effects

```
.card:hover {  
  transform: scale(1.05) translateY(-5px);  
  box-shadow: 0 10px 20px rgba(0,0,0,0.2);  
}
```


d) Image Animations

```
.card img {  
  width: 100%;  
  display: block;  
  transition: transform 0.3s ease;  
}
```

```
.card:hover img {  
  transform: scale(1.1);  
}
```

e) Button Animation (Keyframes)

```
@keyframes bounce {  
  0%, 100% { transform: translateY(0); }  
  50% { transform: translateY(-5px); }  
}
```

```
.card button:hover {  
  animation: bounce 0.5s infinite;  
}
```

Responsive Design 4

- استخدم `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));` → البطاقات تتكيف مع حجم الشاشة
- استخدم `media queries` إذا محتاج تعديل حجم النص أو البطاقات

5 الهدف من التمرين

- تطبيق **Flex/Grid** لتصميم responsive layout
- التعامل مع **Transitions** لتأثير hover سلس
- التعامل مع **Transform & Keyframes** لجعل الحركة جذابة
- تجربة **Colors, Fonts & Box-shadow** لتحسين المظهر
- استخدام **Variables** لتسهيل تغيير ألوان/حجوم البطاقات

Goal:

Use CSS Layout (Flexbox/Grid)

Apply Colors, Fonts & Units

Use Transitions, Transforms, Keyframes Animations

Experiment with Hover effects and interactive UI

1] Project Idea

A page with Cards representing products, articles, or services.

Each card contains:

- Image
- Title
- Short description
- Button or link

Requirements:

- Display cards in a responsive grid or flex layout
- On hover:
 - Slightly scale up the card
 - Show additional info (e.g., description or overlay)
 - Change color or shadow (box-shadow)
 - Optionally add continuous animation or keyframes effects (e.g., small motion for button or product image)

2 HTML Structure

```
<div class="cards-container">  
  <div class="card">  
      
    <h3>Product 1</h3>  
    <p>Short description about the product.</p>
```

```
        <button>Learn More</button>
    </div>
    <div class="card">
        
        <h3>Product 2</h3>
        <p>Short description about the product.</p>
        <button>Learn More</button>
    </div>
    <!-- More cards -->
</div>
```

3 CSS Styling

a) Layout & Grid / Flex

```
.cards-container {
    display: grid;

    grid-template-columns: repeat(auto-fit,
minmax(250px, 1fr));

    gap: 20px;

    padding: 20px;
```

```
}
```

b) Card Base Styles

```
.card {  
    background-color: white;  
    border-radius: 12px;  
    overflow: hidden;  
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);  
    transition: transform 0.3s ease, box-shadow  
0.3s ease;  
}
```

c) Hover Effects

```
.card:hover {  
    transform: scale(1.05) translateY(-5px);  
    box-shadow: 0 10px 20px rgba(0,0,0,0.2);  
}
```

d) Image Animations

```
.card img {  
    width: 100%;  
    display: block;  
    transition: transform 0.3s ease;  
}
```

```
.card:hover img {  
    transform: scale(1.1);  
}
```

e) Button Animation (Keyframes)

```
@keyframes bounce {  
    0%, 100% { transform: translateY(0); }  
    50% { transform: translateY(-5px); }  
}
```

```
.card button:hover {  
    animation: bounce 0.5s infinite;
```

}

4 Responsive Design

- Use `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));` → cards adapt to screen size
- Use media queries if you need to adjust text size or card dimensions

5 Exercise Goal

- Apply Flex/Grid to build a responsive layout
- Use Transitions for smooth hover effects
- Use Transform & Keyframes for attractive motion
- Experiment with Colors, Fonts & Box-shadow for better appearance
- Use Variables to easily change card colors/sizes

Task

Task: Responsive Website with Layout & Animations

Layout 1

- استخدام **Grid** أو **Flexbox** لترتيب العناصر
- موقع يحتوي على **Header, Main Content, Footer**
- تصميم **Sections 5-3** داخل Main Content (مثل, Hero, Features, Cards, Testimonials)

Responsive Design 2

- يجب أن يتكيف الموقع مع **Mobile, Tablet, Desktop**
- استخدام **Media Queries** لتعديل التصميم عند breakpoints مختلفة
- الصور والخطوط يجب أن تكون **مرنة (responsive)**

CSS Animations & Transitions 3

- تطبيق **Hover effects** على Buttons, Cards, Images
- استخدام **Transform** لتكبير / تصغير / تحريك العناصر عند التفاعل

- إنشاء **Keyframes Animation** لعناصر متحركة (مثل, loader, bouncing elements, floating icons)
- جميع الحركات يجب أن تكون **smooth** و non-distracting

Styling 4

- استخدام **CSS Variables** للألوان والخطوط والمسافات
- استخدام **Typography & Colors** متناسقة وجذابة
- إضافة **Box-shadow, Border-radius** لتحسين المظهر

Content 5

- الموقع يحتوي على (Text & Images) **Placeholder content**
- العناوين واضحة وسهلة القراءة
- عناصر تفاعلية واضحة (مثل Buttons, Links, Cards)

(Bonus / Advanced (Optional 6

- **Interactive Card Layout** مع hover overlay
- **Animated Hero Section** عند تحميل الصفحة
- **Light/Dark theme toggle** باستخدام CSS Variables

Deliverables - التسليمات

- ملفات **HTML, CSS, Assets** منظمة
- موقع **Responsive & Fully Functional**
- تطبيق **Animations** بشكل عملي ومرئي
- **Code clean & structured** لتسهيل القراءة والصيانة

1 Layout

Use Grid or Flexbox to arrange elements

Website includes Header, Main Content, Footer

Design 3-5 Sections inside Main Content (e.g., Hero, Features, Cards, Testimonials)

2 Responsive Design

The website must adapt to Mobile, Tablet, Desktop

Use Media Queries to adjust design at different breakpoints

Images and fonts must be flexible (responsive)

3 CSS Animations & Transitions

Apply Hover effects on Buttons, Cards, Images

Use Transform to scale / move elements on interaction

Create Keyframes Animation for moving elements (e.g., loader, bouncing elements, floating icons)

All animations should be smooth and non-distracting

4 Styling

Use CSS Variables for colors, fonts, and spacing

Use consistent and attractive Typography & Colors

Add Box-shadow, Border-radius to enhance appearance

5 Content

Website includes Placeholder content (Text & Images)

Headings are clear and easy to read

Interactive elements are clear (e.g., Buttons, Links, Cards)

6 Bonus / Advanced (Optional)

Interactive Card Layout with hover overlay

Animated Hero Section on page load

Light/Dark theme toggle using CSS Variables

Deliverables

- Organized HTML, CSS, and Assets files

- Responsive & Fully Functional Website
- Practical and visible implementation of Animations
- Clean & structured code for easy reading and maintenance